



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A Major Project Report
On
Automated Marking and Feedback System for Multi-Step Algebraic Math
Solution using Symbolic Reasoning and DeepSeekMath**

Submitted By:

Jesis Upadhayaya	(46316)
Prajwal Chaudary	(46327)
Purushottam Gajurel	(46331)
Sagar Bikram Adhikari	(46336)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

May, 2026



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A Major Project Report
On
Automated Marking and Feedback System for Multi-Step Algebraic Math
Solution using Symbolic Reasoning and DeepSeekMath**

Submitted By:

Jesis Upadhayaya	(46316)
Prajwal Chaudary	(46327)
Purushottam Gajurel	(46331)
Sagar Bikram Adhikari	(46336)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

In partial fulfillment for the award of the Bachelor's Degree in Computer Engineering.

Under the Supervision of
Asst. Prof. Suwarna Lingden

May, 2026

DECLARATION

We hereby declare that the report of the project entitled "**Automated Marking and Feedback System for Multi-Step Algebraic Math Solution using Symbolic Reasoning and DeepSeekMath**" which is being submitted to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in **Computer Engineering**, is a bonafide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only author of this complete work and no sources other than the listed here have been used in this work.

Jesis Upadhayaya (THA078BCT017) _____

Prajwal Chaudary (THA078BCT028) _____

Purushottam Gajurel (THA078BCT032) _____

Sagar Bikram Adhikari (THA078BCT037) _____

DATE: May, 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a major project work entitled “**Automated Marking and Feedback System for Multi-Step Algebraic Math Solution using Symbolic Reasoning and DeepSeekMath**” submitted by **Jesis Upadhayaya, Prajwal Chaudary, Purushottam Gajurel and Sagar Bikram Adhikari** in partial fulfillment for the award of Bachelor’s Degree in Computer Engineering. The Project was carried out under special supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor’s degree of Computer Engineering.

Supervisor: **Asst. Prof. Suwarna Lingden**

Department of Electronics and Computer Engineering, Thapathali Campus

External Examiner: **Dr. Pradip Paudyal**

Deputy Director, Nepal Telecommunications Authority, Kantipath, Jamal, Kathmandu

Head of Department: **Er. Umesh Kanta Ghimire**

Department of Electronics and Computer Engineering, Thapathali Campus

May, 2026

COPYRIGHT

The authors have agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the authors have agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the authors of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGMENT

We would like to extend our heartfelt appreciation to the Institute of Engineering, Tribhuvan University, for including the major project as a component of the **Bachelor's degree in Computer Engineering**. This chance has been crucial in enhancing our technical and research skills. We are grateful to the **Department of Electronics and Computer Engineering, Thapathali Campus**, for their assistance in project selection, guidance, and for creating an environment that promotes innovation and practical learning.

We are grateful to our Supervisor **Asst. Prof. Suwarna Lingden**, for guiding us in every step of our project. Also, We would like to extend our gratitude to all the teachers, friends, and contributors, both direct and indirect, whose insights, encouragement, and suggestions played a crucial role in our journey. Lastly, we want to express our sincere gratitude to all our team members for their commitment, effective teamwork, and shared vision.

Sincerely,

Jesis Upadhyaya (46316)

Prajwal Chaudary (46327)

Purushottam Gajurel (46331)

Sagar Bikram Adhikari (46336)

ABSTRACT

For a long time, human graders have manually checked and marked answer sheets, specially those with multi-step solutions. Manual marking of handwritten solutions is time-consuming, prone to inconsistency, and difficult to scale for large volumes. While automating the assessment of final answers or objective-type questions is relatively simple, evaluating handwritten, step-by-step solutions creates a complex challenge. This project introduces an automated marking system that uses Optical Character Recognition (OCR), symbolic reasoning, and a fine-tuned DeepSeekMath model to evaluate and provide feedback on multi-step algebraic problems' solution. This system can evaluate the algebraic problems from basic (grades 1-8) and secondary education (grades 9-10) with mainly three topics- Algebraic simplification, Algebraic equations and Logarithmic expressions with varying difficulty levels- Easy, Medium and Hard. OCR module with mathematical corpus, Mathpix extracts handwritten or printed solutions from raw images and then converts them in a LaTeX format. Pattern matching is done via RegEx during LaTeX preprocessing which are parsed and symbolically analyzed using expression tree-based symbolic Computer Algebra System (CAS)- SymPy to assess procedural accuracy and mathematical correctness. 219 Seed Rubrics dataset with question rubric pairs are collected manually and by augmentation, total of 2797 JSON format samples were obtained. DeepSeekMath-7B model is fine-tuned on Rubric dataset with 80:10:10 splitting that generates rubrics for the same question with multiple equivalent forms and the min-max normalized marking weight of each critical steps. Another comparison model compares the generated rubric and output of the symbolic reasoning engine to provide final marks based on Total Rubric Score as a evaluation metric which aggregates StepwiseScore, MilestoneScore and FinalAnswerScore metrices. This system combines symbolic reasoning and a large language model, enabling partial marking, detecting logical errors, and generating feedback for critical steps. Moreover, the system can be scaled for variety of other question categories with multiple solutions, it may miss subtle conceptual errors that humans catch.

Keywords: Auto-grading, Algebraic Problems, OCR, Mathematical Reasoning CAS, Symbolic Reasoning, DeepSeekMath, Rubric Based Marking.

Table of Contents

DECLARATION	i
CERTIFICATE OF APPROVAL	ii
COPYRIGHT	iii
ACKNOWLEDGMENT	iv
ABSTRACT	v
List of Figures	xi
List of Tables	xii
Listings	xiii
List of Abbreviations	xiv
1 INTRODUCTION.....	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Project Objectives	2
1.5 Project Scope and Applications	2
1.6 Project Limitations	3
1.7 Report Organization.....	3
2 LITERATURE REVIEW	5
2.1 Mathematical Language Processing.....	5
2.2 Rule-Based Grading of Math Answers	5
2.3 Grade Like a Human	5
2.4 Fine-Tuned T5 for Automated Grading of Quadratic Equations	6
2.5 Integrating NLP and OCR for Automated Evaluation of Descriptive An- swers	6
2.6 Large Language Models for Mathematical Reasoning	7
2.7 Let's Verify Step by Step	8
3 REQUIREMENTS ANALYSIS.....	9
3.1 Hardware Requirements	9
3.2 Software Requirements.....	9
3.2.1 Python	9

3.2.2	React.....	9
3.2.3	Flask	9
3.2.4	Mathpix SDK.....	10
3.2.5	Git/Github	10
3.2.6	Pandas	10
3.2.7	PyTorch	10
3.2.8	HuggingFace	10
3.2.9	Google Colab.....	10
3.3	Feasibility Analysis	11
3.3.1	Technical Feasibility	11
3.3.2	Budget Feasibility	11
3.3.3	Scheduling Feasibility.....	11
4	SYSTEM ARCHITECTURE AND METHODOLOGY	12
4.1	System Block Diagram.....	12
4.1.1	Input interface	12
4.1.2	OCR Module	13
4.1.3	Pre-processing Module.....	14
4.1.3.1	Working Mechanism of Regular Expressions	15
4.1.4	Mathematical Parser	17
4.1.5	Symbolic Reasoning Engine.....	18
4.1.6	Finetuned DeepSeekMath Model as Rubrics Generator	19
4.1.7	LLM-based Marking and Feedback.....	19
4.1.8	Output Interface.....	19
4.2	System Flow Diagram	20
4.3	Working Principle	20
4.3.1	Mathpix OCR	20
4.3.1.1	Encoder–Decoder Architecture	20
4.3.1.2	Output Formats	21
4.3.2	Data Collection	22
4.3.3	Data Preprocessing / Preparation	23
4.3.3.1	OCR-Based Text Extraction	23
4.3.3.2	OCR Noise and Error Analysis	23
4.3.3.3	Manual Verification and Content Filtering	24
4.3.3.4	Dataset Completion and Quality Control.....	24

4.3.3.5	Stepwise Mathematical Segmentation.....	24
4.3.3.6	LaTeX Transformation	25
4.3.3.7	Dataset Structuring and Annotation	26
4.3.3.8	Marks Normalization	27
4.3.3.9	Data Augmentation	28
4.3.4	Exploratory Data Analysis	30
4.3.5	DeepSeekMath-7B Model.....	31
4.3.5.1	Model Overview	32
4.3.5.2	Architecture Details	32
4.3.5.3	Tokenization	41
4.3.5.4	Training Methodology	42
4.3.5.5	Mathematical Reasoning Capabilities	44
4.3.5.6	Chain-of-Thought Reasoning	45
4.3.6	Symbolic Reasoning Engine: SymPy Core Architecture	47
4.3.6.1	Core Design and Expression Representation	47
4.3.6.2	Parsing and Canonicalization.....	48
4.3.6.3	Rule-Based Transformation and Simplification.....	49
4.3.6.4	Equation Solving and Logical Verification.....	49
4.3.6.5	Stepwise Validation in Auto-Grading	49
4.3.6.6	Integration with DeepSeekMath	49
4.3.7	Optimization.....	50
4.3.7.1	Momentum.....	50
4.3.7.2	Adagrad.....	50
4.3.7.3	RMSProp.....	51
4.3.7.4	Adam	51
4.3.8	Activation Function	51
4.3.8.1	Gaussian Error Linear Unit	51
4.3.8.2	Swish Activation Function	52
4.3.9	Evaluation Metrics	53
4.3.9.1	OCR Metrics	53
4.3.9.2	Answer Marking Metrics	54
4.3.9.3	Model Evaluation Metrics.....	55
5	IMPLEMENTATION DETAILS	57
5.1	Mathpix OCR Integration.....	57
5.1.1	Loading environment variables	57
5.1.2	Initialization of the Mathpix client	57

5.1.3	Calling Mathpix API for OCR	57
5.2	Preprocessing Pipeline	58
5.2.1	Algorithm	58
5.2.2	Flowchart	59
5.3	Mathematical Parser	59
5.3.1	Algorithm	60
5.3.2	Flowchart	61
5.4	Symbolic Reasoning Engine	61
5.4.1	Algorithm	62
5.4.2	Flowchart	64
5.5	DeepSeekMath Model Training	66
5.5.1	Dataset Preparation	66
5.5.2	Training Configuration	66
5.5.3	Loss Function	67
6	RESULT AND ANALYSIS	68
6.1	Frontend Development	68
6.1.1	Output from Symbolic Reasoning Engine	71
6.2	Train/Validation loss curve	77
6.3	Model Results	78
6.4	Output obtained from Marking and Feedback Engine	79
6.5	Analysis of Results	84
7	CONCLUSION	85
8	APPENDICES	87
Appendix A	: Gantt Chart	87
Appendix B	: Project Budget	88
Appendix C	: Code Snippets	89
Appendix D	: Details of Rubrics Dataset	93
Appendix E	: Response from Marking and Feedback Engine	95
Appendix F	: Plagiarism Report Summary Page	98
Appendix G	: Student and Supervisor Consultation Forms	101
	References	105

List of Figures

Figure 4-1	System Block Diagram	12
Figure 4-2	OCR Module Working via Mathpix	14
Figure 4-3	Mathematical Parser Working	18
Figure 4-4	System Flow Diagram	20
Figure 4-5	Block diagram of Mathpix OCR Architecture	21
Figure 4-6	Original SEE Sample Question along with its Marking Rubrics published by NEB	22
Figure 4-7	Distribution of Dataset Across Different Topics	30
Figure 4-8	Distribution of Dataset Across Difficulty levels per Topics	31
Figure 4-9	Block Diagram of Transformer Architecture	33
Figure 4-10	Transformer Block	34
Figure 4-11	Attention Heads	36
Figure 4-12	Multi-layer Perceptron	38
Figure 4-13	LayerNorm	40
Figure 4-14	Block Diagram of LoRA reparametrization	44
Figure 4-15	Parse tree for $\text{expr} = x*y + 2$	48
Figure 4-16	GELU Activation Function	52
Figure 4-17	SiLU Activation Function	53
Figure 5-1	Preprocessing Flowchart	59
Figure 5-2	Mathematical Parser Flowchart	61
Figure 5-3	symbolic Reasoning Engine Flowchart	64
Figure 5-4	Step Validation Module Flowchart	65
Figure 6-1	Home page	68
Figure 6-2	Upload Page	69
Figure 6-3	Rendered latex	70
Figure 6-4	Preprocessed output	70
Figure 6-5	Parser output	71
Figure 6-6	Symbolic reasoning engine output	71
Figure 6-7	Symbolic reasoning engine's result for a sample image with an algebraic expression with an invalid step and error propagation	72
Figure 6-8	Symbolic reasoning engine's result for a sample image with an algebraic expression with all correct solution steps	73
Figure 6-9	Symbolic reasoning engine's result for a sample image with a logarithmic expression with an incorrect final answer	74

Figure 6-10 Symbolic reasoning engine's result for a sample image with a logarithmic expression with all correct intermediate steps as well as the final answer	75
Figure 6-11 Symbolic reasoning engine's result for a sample image with an algebraic expression with some incorrect steps	76
Figure 6-12 Symbolic reasoning engine's result for a sample image with an algebraic expression with correct steps	77
Figure 6-13 Loss Curve	78
Figure 6-14 A sample image with all solution steps correct receives full marks along with feedback.	80
Figure 6-15 A sample image with incorrect intermediate solution steps receives partial marks along with feedback.	81
Figure 6-16 A sample image with incorrect intermediate logarithmic solution steps receives partial marks along with feedback.	82
Figure 6-17 A sample image with incorrect intermediate solution steps receives partial marks along with feedback.	83
Figure 8-1 Summary Page of Plagiarism Report showing similarity index	98
Figure 8-2 Plagiarism Report showing Top sources	99
Figure 8-3 AI writing Overview	100
Figure 8-4 Student and Supervisor Consultation Form (Part-A).....	101
Figure 8-5 Student and Supervisor Consultation Form (Part-B).....	102

List of Tables

Table 3-1	Hardware Specification	9
Table 4-1	DeepSeekMath-7B model dimensions	35
Table 6-1	New questions provided to the fine-tuned DeepSeekMath model.	78
Table 6-2	Generated rubrics for the new questions after fine-tuning.	79
Table 8-1	Gantt Chart.....	87
Table 8-2	Project Budget.....	88

Listings

1	Raw image to LaTeX Transformation	25
2	Example of step-by-step reasoning	45
3	Multiple Solution Paths	45
4	Example of Standard Prompting	46
5	Example of Chain-of-Thought Prompting	46
6	Example of Rubric Generation	46
7	Example of Step-by-Step Evaluation	47
8	Loading Environment variables	89
9	Initializing Mathpix Client	89
10	Calling Mathpix API for OCR	89
11	LaTeX Preprocessing Module	89
12	Mathematical Parser Module	90
13	Check Step Validity Function	90
14	Symbolic Reasoning Engine	91
15	Response from Marking and Feedback Engine	95

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BLE	Basic Level Examination
CAA	Computer-Assisted Assessment
CAS	Computer Algebra System
CDC	Curriculum Development Center
CER	Character Error Rate
CNN	Convolution Neural Networks
DL	Deep Learning
EQ	Algebraic Equation
GB	Giga Bytes
GCE	General Certificate of Education
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
GSM8K	Grade School Math 8K Dataset
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
LHS	Left Hand Side
LLM	Large Language Model

LOG	Logarithm and Indices
LoRA	Low-Rank Adaptation (used in model fine-tuning)
MATH	Mathematics benchmark dataset
ML	Machine Learning
MathLP	Mathematical Language Processing
MLP	Multi-Layer Perceptron
MMD	Mathpix Markdown
NEB	National Examination Board
NLP	Natural Language Processing
OCR	Optical Character Recognition
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantized Low-Rank Adaptation
RAM	Random Access Memory
RegEx	Regular Expressions
RHS	Right Hand Side
RMS	Root Mean Squares
SDK	Software Development Toolkit
SEE	Secondary Education Examination
SIM	Algebraic Simplification
SiLU	Sigmoid Linear Unit
SME	Subject Matter Expert
STEM	Science, Technology, Engineering, and Mathematics

T5	Text-to-Text Transfer Transformer
TPU	Tensor Processing Unit
TrOCR	Transformer-based Optical Character Recognition
UI	User Interface
XML	eXtensible Markup Language

1 INTRODUCTION

1.1 Background

Assessment plays a crucial role in the teaching-learning process, providing valuable insights into students' comprehension of a particular subject. In the context of mathematics education, assessments are used not only to evaluate final answers but also to grade the logical flow and correctness of each intermediate step. Nevertheless, the process of conducting and evaluating these assessments manually is labor-intensive, repetitive, and frequently susceptible to human bias and mistakes. Teachers are responsible for creating question papers, establishing marking schemes, and manually evaluating lengthy, handwritten scripts—all of which add up to a substantial workload. Automating the evaluation of objective or multiple-choice questions is relatively simple, but automating the evaluation of stepwise mathematical problem-solving is a complex challenge [1, 2].

Computer-assisted assessment (CAA) is a growing field of research that investigates the possibilities of intelligent systems in evaluating student learning. Although significant advancements have been made in evaluating short and long-form natural language responses using keyword-based methods, these techniques are not suitable for formal subjects like mathematics, where correctness is determined by logical reasoning and symbolic precision rather than keywords. In mathematics, every step in the process adds to the final score, and partial marks are given to students who demonstrate a logical progression, regardless of whether the final answer is correct. For example, a student who correctly performs all the steps but makes a small mistake in arithmetic might receive a higher score than another student who gets to the final answer by skipping important steps. Even when evaluators share a rubric, grading results may be inconsistent due to different subjective interpretations [3, 4].

To address these problems, our project proposes a system that will imitate the grading behavior of human experts by evaluating the step-by-step solution process in algebraic math problems. The model is able to process handwritten responses, by applying optical character recognition (OCR) to convert them into symbolic representations. It also employs tools such as SymPy [5] to convert mathematical expressions into a symbolic form and makes logical comparisons with marking schemes to award partial marks.

1.2 Motivation

The primary aim of this project is to address the challenges and discrepancies that accompany the manual grading of mathematics papers that need stepwise solutions but are not always graded fairly. The limited time and subjectivity of evaluation often do

not allow evaluators to see intermediate steps and thus make errors in scoring, and do not allow them to recognize partial understanding. As students, we have also felt the frustration of losing points when we were not able to identify exactly why we lost those points, and we knew that existing systems are not a reliable way to evaluate complex mathematical reasoning consistently. Our aim is to create a trustworthy, transparent, and scalable system that enables accurate grading and assists students and teachers in enhancing the quality of learning and assessment via automated, step-by-step solution evaluations using OCR, symbolic reasoning, and AI.

1.3 Problem Statement

Manual assessment of mathematics assignments, especially in the mathematical algebra field, is time-consuming and subjective, prone to human errors, and lacks uniformity. Multi-step solutions are difficult to evaluate for human graders because of limited time and different grading criteria. Most of the automated tools available today only check the final results and cannot check the correctness of each step. Also, existing systems have trouble with handwritten responses and are not prepared to handle multiple correct solution approaches. This results in partial scoring and limited feedback for students, which hampers their educational growth. So we are trying to solve this problem by using an LLM-based grader.

1.4 Project Objectives

- To create an automatic marking system for multi-step handwritten mathematical solutions.

1.5 Project Scope and Applications

Checking mathematics answer sheets manually is time-consuming, inconsistent, and subject to individual bias. This system can help schools, examination boards, and students in marking structured algebra problems more smoothly. This system allows us to make the whole evaluation process more consistent, faster, and clearer for everyone. The system can be used to assist in grading during exams, providing a reliable and scalable solution, especially when dealing with a large number of answer sheets.

From a research perspective, this project contributes to the growing area of NLP in educational AI. Its flexible design, stepwise logic interpretation, and feedback generation can serve as a starting point for researchers who want to explore other areas of mathematics or develop multilingual mathematical assessments.

1.6 Project Limitations

- Currently limited to a few categories of questions only.
- Your handwriting should be reasonably clear.
- Supports only English and a few mathematical notations.
- May overlook subtle conceptual errors that humans can catch.

1.7 Report Organization

The report is divided into eight sections, each dealing with a separate aspect of the project.

Section 1: Introduction presents the background and motivation to develop an automated marking and feedback system for multi-step algebraic problems. It describes the problem statement, project objectives, scope, applications, and limitations.

Section 2: Literature Review surveys relevant existing work comprising mathematical language processing, rule-based grading, LLM-based evaluation frameworks and step-by-step verification approaches. It highlights the gaps this project aims to address.

Section 3: Requirements Analysis describes the hardware and software requirements to build and run the system and a feasibility analysis covering technical, budgetary, and scheduling aspects.

Section 4: System Architecture and Methodology describes the overall system design, including the block diagram, flow diagram, and working principles of each component; the OCR module, preprocessing pipeline, mathematical parser, symbolic reasoning engine (SymPy), DeepSeekMath-based rubric generator, LLM-based marking module, and output interface.

Section 5: Implementation Details discusses the specific implementation of each system component, including the integration of Mathpix OCR, the symbolic reasoning pipeline, the fine-tuning of DeepSeekMath-7B, and the web-based user interface.

Section 6: Result and Analysis discusses the performance of the system based on the defined metrics such as OCR accuracy, stepwise scoring, milestone scoring, final answer scoring and the Total Rubric Score. It discusses the implications and results of the experiments.

Section 7: Conclusion summarizes the results of the project, reflects on what was achieved relative to stated objectives, and suggests potential directions for future work.

Section 8: Appendices contains additional material, including code listings, samples of datasets, and other technical details referenced throughout the report.

2 LITERATURE REVIEW

2.1 Mathematical Language Processing

Mathematical Language Processing (MathLP) is a framework designed to automatically grade open-response mathematical questions, unlike multiple-choice inputs or binary grading schemes (correct/incorrect) used in STEM courses. It converts each solution into a numerical feature denoted by symbolic mathematics, using algorithms like k-means and Bayesian nonparametric methods. These feature vectors were clustered into correct, incorrect, and partially correct solutions. When a human assigns a grade to a solution in a cluster, the rest of the solutions in the same cluster are automatically graded. MathLP assumes typed input and cannot handle handwritten math. MathLP does not check whether mathematical transformations are valid; it relies on statistical similarity instead of symbolic correctness. The model is older than transformer-based architectures and does not leverage deep learning or LLMs. Due to this, it can only afford limited adaptability and performance in more complex evaluation tasks [1].

2.2 Rule-Based Grading of Math Answers

Kadupitiya et al. [2] developed an automated grading system for GCE (General Certificate of Education) O/L (Ordinary Level) mathematical word problems in Sri Lanka. The system uses a regular expression-based information retrieval approach to validate the expressions in the answers, achieving 99.8% accuracy on 500 answers using regular expressions and XML rubrics. They used three modules, i.e., the expression validation module, the units validation module, and the automatic grading module. The expression validation module validates student answers, the unit validation module validates the units of expressions using the Levenshtein (edit) distance method, whereas the automatic grading module awards full or partial marks to student answers strictly based on the marking rubric. It handles multi-step answers but lacks NLP, limiting feedback. The rigid expression validation misses equivalent solutions, and the XML representation of solutions has affected scalability.

2.3 Grade Like a Human

It has been difficult to automate the grading of open-ended student answers. This is mainly because of the subjective nature and context dependency of human grading. The recent work by W. Xie *et al.* [3] introduces *Grade-Like-a-Human (GLAH)*, a large language model (LLM)-based framework that addresses these challenges through a three-stage pipeline: rubric generation, rubric-guided grading, and post-grading calibration.

The first component of the GLAH system tackles the issue of rubric unavailability or

variability. The system does not rely on traditional, human-authored rubrics, but automatically generates rubrics using LLMs and exemplary responses. This method uses "in-context learning" to determine how to grade responses. It means it can identify the relevant grading criteria without having to specify them for each task. Then, the model generates a set of scoring rules that can also be used for open-ended answers, boosting scalability.

For grading, the model uses the previously created rubrics and compares them with the student answers, similar to how teachers check answer sheets. Instead of atomic checking (giving one overall score based on similarity), it checks the answer against each part of the rubric separately.

2.4 Fine-Tuned T5 for Automated Grading of Quadratic Equations

The fine-tuned T5 model is used for automating the grading of quadratic equation problems. The dataset is manually curated, with unique quadratic equation solutions, with the help of subject matter experts (SMEs), and used for training and testing [6].

The study compares the results from traditional machine learning (ML) models and deep learning (DL) models. Initially, the pretrained T5 model gave unsatisfactory results due to the limited dataset size. Fine-tuning the T5 model significantly improved the performance, reducing error by 70% and increasing the R2 value to 0.98. Ensemble ML models like XGBoost and CatBoost outperformed DL models such as LSTM and BiLSTM, which struggled due to the dataset's size [4].

However, the model's usage is limited to typed textual inputs and a limited scope of problems (quadratic equations). Handwritten responses are not addressed, which is common in educational environments. Also, the system only evaluates the final answer instead of performing step-by-step analysis, which is unfair for partially correct answers.

2.5 Integrating NLP and OCR for Automated Evaluation of Descriptive Answers

In the 2024 study, P. Deepak *et al.* [7] propose an automated system that combines Optical Character Recognition (OCR) and Natural Language Processing (NLP) to evaluate handwritten descriptive answer scripts. The OCR converts the handwritten responses into machine-readable form. After that, NLP techniques are applied for text analysis, such as semantic content, coherence, and grammatical accuracy of the responses. The state-of-the-art language models interpret the depth of understanding in student answers, similar to the evaluation performed by human graders.

However, the system is not able to work with complex math symbols and problems, even though it can easily handle text-based answers. Also, it fails to evaluate step-by-step reasoning, which is crucial in grading math problems.

2.6 Large Language Models for Mathematical Reasoning

Large Language Models (LLMs), originating from the transformer architecture introduced by Vaswani et al. [8], have been widely used for mathematical reasoning. These models use probabilistic methods to generate responses.

In recent times, we can observe the emergence of specialized LLMs tailored for mathematical tasks. Microsoft’s Phi-2, a compact 2.7 billion-parameter model, demonstrates that smaller models, when trained on high-quality datasets, can rival or even surpass larger counterparts in reasoning tasks. Phi-2’s performance underscores the importance of data quality and training methodologies in enhancing model capabilities [9].

Similarly, MathCoder introduces a method to fine-tune open-source language models for modeling and deriving mathematical equations. It achieved state-of-the-art scores among open-source LLMs on the MATH (45.2%) and GSM8K (83.9%) datasets [10].

Forootani’s survey [11] provides a better insight into the evolution of AI-driven mathematical problem-solving, tracing developments from early statistical methods to contemporary transformer-based architectures. LLMs have shown good performance in arithmetic operations, complex reasoning, theorem proving, and structured symbolic computation. Their application extends to optimization and control frameworks, including mixed-integer programming, linear quadratic control, and multi-agent optimization strategies. Enhancement techniques such as Chain of Thought prompting, instruction tuning, and tool-augmented methods have been employed to bolster LLMs’ problem-solving performance.

Recently, methods based on chain-of-thought prompting have been very effective with LLMs in assessing the responses of students in mathematics and scientific fields. It enables models to decompose a task into manageable steps. Studies on improving LLM-based short-answer grading using retrieval-augmented generation have indicated substantial gains when scaling up with large knowledge bases along with LLM-powered logical reasoning [12, 13].

Despite these advancements, certain challenges are still observed. LLMs often struggle with numerical precision, logical consistency, and proof verification. Emerging trends like hybrid neural-symbolic reasoning, structured prompt engineering, and multi-step

self-correction are being studied to address these limitations. Future research directions emphasize improving interpretability, integrating domain-specific solvers, and enhancing AI-driven decision-making.

2.7 Let's Verify Step by Step

Recent advancements in LLMs have shown significant improvement in their ability to solve complex problems involving multi-step reasoning, particularly in domains such as mathematics and logical inference. Techniques such as Chain-of-Thought prompting have enabled models to explicitly generate intermediate reasoning steps, which has helped significantly improve performance on reasoning benchmarks. However, despite these improvements, state-of-the-art models continue to suffer from logical inconsistencies and hallucinations, where a single incorrect reasoning step can invalidate an otherwise plausible solution.

To enhance the reliability of reasoning models, researchers have explored training reward models using human feedback. Two primary supervision strategies have emerged in this context: outcome supervision and process supervision. Outcome supervision provides feedback only on the final answer produced by a model, whereas process supervision evaluates the correctness of each intermediate reasoning step.

Cobbe et al. [14] (2023) extended this line of research by conducting a large-scale empirical comparison between outcome-supervised reward models (ORMs) and process-supervised reward models (PRMs) on the more challenging MATH dataset. Their study showed that process supervision significantly outperforms outcome supervision when scaled with sufficient human feedback. The process-supervised model achieved a solve rate of over 78% on a representative subset of the MATH test set, clearly surpassing outcome-supervised models and majority-voting baselines. The main contribution of the research is the creation of the PME 800k dataset containing approximately 800,000 step-level human annotations across thousands of math problem solutions. These annotations enable fine-grained supervision by identifying the precise step where reasoning errors occur. This granular feedback improves credit assignment during training, allowing models to distinguish between partially correct and fundamentally flawed solutions more effectively than outcome supervision.

The models also demonstrated active learning, resulting in improved data efficiency in process supervision. Greater focus on reasoning rather than the final answer leads to more reliable, interpretable, and aligned reasoning models.

3 REQUIREMENTS ANALYSIS

3.1 Hardware Requirements

GPUs are important part of our project. They are required for the training and inference of deep learning models like DeepSeekMath. These models are compute-intensive and Google Colab Pro is the most suitable platform for our purpose. It can provide necessary GPU access and runtime support for our experiments.

Google Colab Specifications: The hardware configuration for Colab is shown in **Table 3-1**.

Table 3-1: Hardware Specification

Hardware Components	Details
CPU	Intel Xeon 2-core (2 vCPU)
GPU Model	NVIDIA Tesla V100, T4, or P100 (priority access)
Number of GPUs	1
CUDA Cores Per GPU	5120 (V100), 2560 (T4), 3584 (P100)
VRAM (GPU)	16 GB (V100/P100) or 15 GB (T4)
RAM	Up to 52 GB
Storage	Around 150 GB SSD

3.2 Software Requirements

3.2.1 Python

Python is a high-level, open-source programming language widely used in data science, machine learning, and deep learning applications. Python serves as the primary programming language for developing the entire system, including data pre-processing, model training, and integration.

3.2.2 React

React is a JavaScript library used to build fast, interactive User interfaces (Frontend). It helps in creating reusable components and updating the webpage efficiently without reloading it. In this project, React is used to develop the frontend interface where users can upload handwritten solutions and easily view the recognized expressions and feedback generated by the system.

3.2.3 Flask

Flask is a lightweight Python web framework used to build web applications and APIs. It is employed for making the web server and processing HTTP responses and requests.

Flask offers us the basis for our web application and enables us to build routes, render templates, and process user input. It is necessary to serve the web interface where users can upload images of handwritten mathematical solutions and see inference results. It also supports easy integration of deep learning models with the backend.

3.2.4 Mathpix SDK

Mathpix Python SDK (mpxpy) is the official Python client for accessing Mathpix API services. It offers a simple and flexible way to integrate OCR, document conversion, and Mathpix Markdown (MMD) processing into Python applications. It is used to seamlessly integrate the Mathpix API into our applications.

3.2.5 Git/Github

Git is a version control system, and GitHub is a cloud-based platform for hosting and sharing code repositories. Git/GitHub is used for version control and collaboration among team members.

3.2.6 Pandas

Pandas is a data manipulation and analysis library used for handling and processing data in tabular form. Provides powerful data structures and functions for manipulating and analyzing data.

3.2.7 PyTorch

PyTorch is an open-source deep learning framework for developing deep learning models. In this system, PyTorch is used to implement and execute deep learning models required for handwritten expression recognition and related tasks.

3.2.8 HuggingFace

HuggingFace is an open-source platform and ecosystem that provides pre-trained machine learning models, datasets, and tools for building and deploying AI applications. In this project, Hugging Face is used to integrate models that help generate meaningful explanations and feedback for mathematical solutions.

3.2.9 Google Colab

Google Colab is a cloud-based computing platform that offers free GPU and TPU access, making it ideal for deep learning model training. It is compatible with Jupyter notebooks and well integrated with Google Drive, making data storage and sharing convenient. Colab is utilized in this project for fine-tuning LLMs and running infer-

ence.

3.3 Feasibility Analysis

3.3.1 Technical Feasibility

The availability of established frameworks and pre-trained models makes the project technically feasible. The system will use Vision Transformer-based OCR to convert handwritten responses into LaTeX (digital mathematical expressions). The backend logic integrates symbolic solvers such as SymPy with fine-tuned lightweight LLMs (e.g., MathCoder or DeepSeekMath) to evaluate each step. These components can be easily deployed on platforms such as Google Colab Pro or Kaggle GPUs. The frontend will be implemented using React or Next.js to provide an interactive and user-friendly user interface. Flask or FastAPI will manage backend orchestration process. These technologies are lightweight and scalable which supports technical sustainability and extensibility in future versions.

3.3.2 Budget Feasibility

The economic feasibility of the project is favourable to the team. The main expenses involved are for computing resources, such as Colab Pro and cloud storage, costing a total of around NPR 13,000. The project uses open-source models and libraries, such as HuggingFace, SymPy, and OCR. It will probably utilize the Mathpix API as well, to serve as a fallback in case the OCR quality provided by TrOCR is not adequate, especially when dealing with complex handwritten math equations. The development work is done on a volunteer basis because this project is part of students' research efforts. The availability of academic sources and free infrastructure services helps in reducing costs as well.

3.3.3 Scheduling Feasibility

The project is scheduled to be undertaken within ten months. This will be sufficient time for design, development and testing. In the first two months, the emphasis will be placed on basic activities such as gathering data and pre-processing it and developing the OCR pipeline. The next three months will involve integrating the symbolic solver and refining large language models. Months six and seven will be spent working on the frontend interface and feedback components. The last three months will include testing, fine tuning, and documentation. Through such a schedule, we will be able to manage our workload effectively and identify any challenges early enough.

4 SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 System Block Diagram

The system block diagram for our project is given below in **Figure 4-1**.

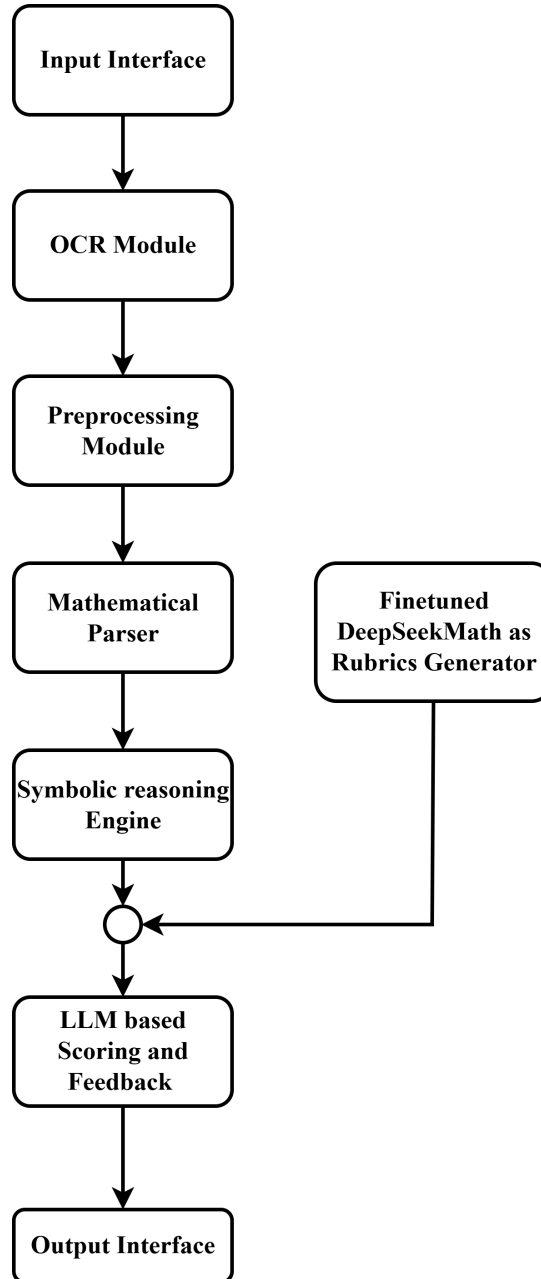


Figure 4-1: System Block Diagram

4.1.1 Input interface

The input interface is the entry point of the system, where user (usually teacher or student) uploads the problem and a corresponding handwritten solution in the form of

image. This block is responsible for collecting the raw input that will be processed, evaluated, and marked by the system.

4.1.2 OCR Module

The Optical Character Recognition (OCR) module is a critical component of our autograding system. Its primary function is to receive an image containing either handwritten or printed mathematics expressions and equations and convert it correctly into a readable format that can be processed by our grading system. Mathpix is specifically designed to recognize a wide variety of complex mathematical content. The input image is fed to an OCR module which reads and extracts the mathematical expressions from the provided image. OCR processes each line of the image and converts it into a clean, structured LaTeX format, which is a standard way to write math using text. Hence OCR transforms visual math into standard form which is used for consistent parsing and evaluation later on.

At first, we tested a Transformer based OCR model called TrOCR [15], which had been trained to recognize both handwritten and printed math content. Since TrOCR's accuracy was not adequate (due to complex, poorly written expressions), we went with Mathpix API to ensure the OCR output is reliable because all of our next steps are fully dependent on the correctness of the OCR results. **Figure 4-2** shows the processing done by OCR module.

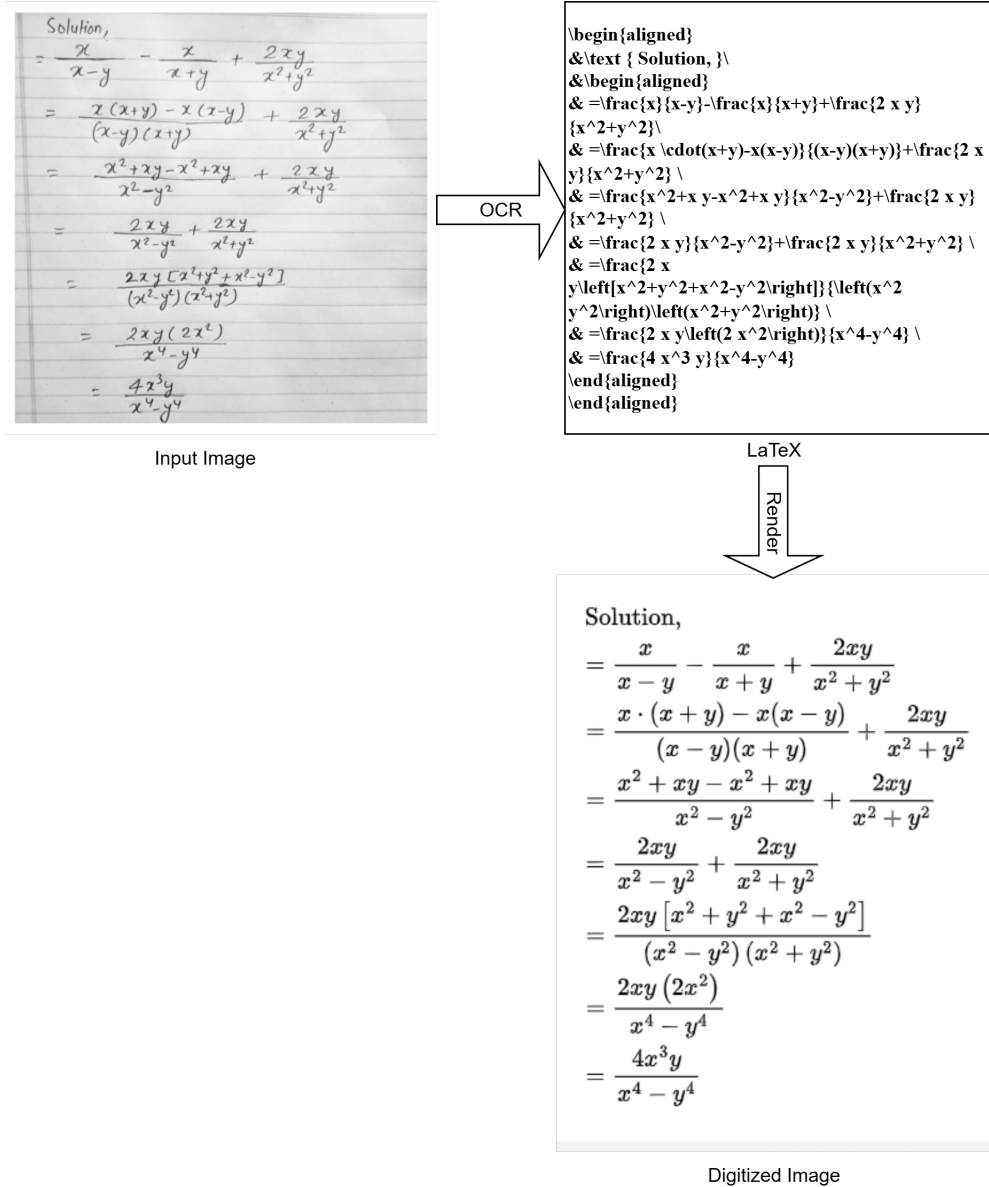


Figure 4-2: OCR Module Working via Mathpix

4.1.3 Pre-processing Module

This module preprocesses the output of the OCR module, which is in LaTeX to format that is suitable to be used by the mathematical parser. The Latex response output from the OCR module is unstructured and difficult to process. So we do necessary cleanups and do pattern matching to get exactly what we need. It extracts all the steps into an array separated by commas. It is done by using Regular expression (Regex) for pattern matching.

4.1.3.1 Working Mechanism of Regular Expressions

At a conceptual level, RegEx operates by defining a search pattern that describes a set of strings. The RegEx engine then scans the input text sequentially and attempts to find substrings that match the defined pattern. This approach is deterministic and reproducible, which is essential for an auto-grading system where consistent interpretation of student solutions is required.

The OCR output, although represented in LaTeX format, often contains unnecessary commands, inconsistent spacing, redundant symbols, and formatting artifacts. Feeding such raw output directly into a mathematical parser can cause parsing errors or incorrect interpretation of solution steps. This output can be filtered selectively by RegEx by matching only the syntactically meaningful components, e.g., mathematical operators, variables, numbers, and step delimiters.

Moreover, regular expression based processing is lightweight and computationally efficient compared to more complex syntactic parsing methods and thus is suitable for real-time or large-scale grading scenarios.

General Working Principle of RegEx

The working mechanism of the RegEx can be summarized as follows:

- i. **Pattern Definition:** A regular pattern expression is used to define the desired structure, e.g. mathematical step, equation, delimiter etc.
- ii. **Sequential Scanning:** The RegEx engine scans the left to right of the OCR generated LaTeX string and matches each character with the pattern .
- iii. **State Matching:** The engine moves from one matching state to another according to the pattern rules until it is a valid match .
- iv. **Extraction and Grouping:** After finding a match, the part of the string matching the pattern is extracted, and capturing groups is used to extract the specific parts of the match.
- v. **Post-processing:** The extracted matches are saved in an ordered array. Each element in the array corresponds to one step in the student's solution.

As an example for a simple OCR-generated LaTeX response for a multi-step solution:

$$\$ \Rightarrow x = 2 \Rightarrow x + 3 = 5 \Rightarrow x = 2\$ \quad (4-1)$$

This representation is visually correct but not directly usable for step-wise grading. The pre-processing module aims to extract each step logically separately.

A representative regular expression pattern for step extraction may be written as:

$$([\Rightarrow]^+) \quad (4-2)$$

This will match any substring that is in between the `\Rightarrow` symbols.

Applying this pattern to the input string, we get the following extracted steps:

- $x = 2$
- $x + 3 = 5$
- $x = 2$

These steps are then stored as a comma-separated array that can be passed directly mathematical parser for symbolic grading and analysis.

Handling Noise and Inconsistencies

Noise, such as additional spaces, repeated characters, or other issues, is often introduced by OCR systems malformed LaTeX directives. RegEx is also used for normalizing such inconsistencies extraction of the step. For example:

- Trimming excess whitespace
- Removal of unsupported LaTeX commands
- Normalizing operators (e.g., converting multiple equality symbols to one form)

This normalization guarantees that semantically equivalent steps are treated the same, which is important for equitable grading.

The RegEx-based pre-processing module bridges perception (OCR) and reasoning (mathematical parsing) by converting unstructured OCR output into a clean, structured sequence of mathematical steps. Any mistakes or ambiguity at this stage would be carried through the system and would affect the grading accuracy. Thus, the deterministic and transparent nature of pattern matching based on RegEx is essential for the reliability of the whole auto-grading framework.

4.1.4 Mathematical Parser

The mathematical parser steps in after the OCR turns the math image into LaTeX. It takes the LaTeX expression and converts it into a format that the computer can understand. It is achieved by using Sympy. The parser breaks the expression into a kind of tree that shows all the parts and how they relate to each other. This clear structure (parse tree) is the key for the system to think through the math, check each step, and mark it accurately.

We have used the `latex2sympy2` library to parse LaTeX steps into a sympy object with some minor tweaks. The `Latex2sympy` library converts the LaTeX solution steps into a Sympy object. Sympy objects are the ones that a computer can truly understand and can perform calculations and operations such as factorize, solve, simplify, equals, etc. The workflow upto mathematical parser is visualized in **Figure 4-3**.

OCR Output

```
\begin{aligned}
&\&\text{Solution, }\backslash
&\begin{aligned}
&=\frac{x}{x-y}-\frac{x}{x+y}+\frac{2xy}{x^2+y^2}\backslash
&=\frac{x\cdot(x+y)-x(x-y)}{(x-y)(x+y)}+\frac{2xy}{x^2+y^2}\backslash
&=\frac{x^2+xy-x^2+xy}{x^2-y^2}+\frac{2xy}{x^2+y^2}\backslash
&=\frac{2xy}{x^2-y^2}+\frac{2xy}{x^2+y^2}\backslash
&=\frac{2xy\left[x^2+y^2+x^2-y^2\right]}{\left(x^2-y^2\right)\left(x^2+y^2\right)}\backslash
&=\frac{2xy\left(2x^2\right)}{x^4-y^4}\backslash
&=\frac{4x^3y}{x^4-y^4}
&\end{aligned}
&\end{aligned}
```

Pre-processing Module

Array of cleaned Latex steps

Math Parser Module

```
-x/(x + y) + x/(x - y) + (2*x*y)/(x**2 + y**2)
(2*x*y)/(x**2 + y**2) + (-x*(x - y) + x*(x + y))/((x - y)*(x + y))
(2*x*y)/(x**2 + y**2) + (-x**2 + x**2 + x*y + x*y)/(x**2 - y**2)
(2*x*y)/(x**2 + y**2) + (2*x*y)/(x**2 - y**2)
(2*x*y*(x**2 + x**2 - y**2 + y**2))/((x**2 - y**2)*(x**2 + y**2))
(2*x*y*2*x**2)/(x**4 + y**4)
(4*x**3*y)/(x**4 - y**4)
```

Parsed SymPy Object

Figure 4-3: Mathematical Parser Working

4.1.5 Symbolic Reasoning Engine

The symbolic reasoning engine checks the logical and mathematical validity of each step in the solution of the student. Using the parsed symbolic representation, it evaluates whether the student's transformation from one step to the next is mathematically correct. It identifies structural or arithmetic errors and determines symbolic equivalence between the student's step and the expected step. This engine produces a structured report summarizing the outcome of the symbolic check, for example, valid and invalid

steps along with error propagation.

4.1.6 Finetuned DeepSeekMath Model as Rubrics Generator

While SymPy handles strict symbolic validation, the LLM handles rubrics generation with a set of valid steps from symbolic reasoning. Deepseek Math model is used to generate a marking rubrics after fine-tuning it on a rubric dataset. The rubrics includes the checkpoints for which certain marks is assigned of the total marks which sum to the total marks.

4.1.7 LLM-based Marking and Feedback

This LLM-based model matches the rubrics from the rubrics generator with the student's valid answer steps, and then if any match is found, then marks are assigned corresponding to the matched rubrics checkpoint. The LLM generates a natural-language explanation of why a step is acceptable or not, offering a more flexible, human-like judgment than symbolic matching alone can provide. For this, we will prompt engineer the model to match the rubric, and if match is found then the marks is assigned corresponding to the matched rubric.

4.1.8 Output Interface

The final block of the system presents the evaluation results to the end-user. It displays scores for each step, highlights mistakes, and shows natural-language feedback generated by the system. The output will then be shown in the web interface. It offers a complete breakdown of student performance, supporting teachers in grading and helping students understand their mistakes.

4.2 System Flow Diagram

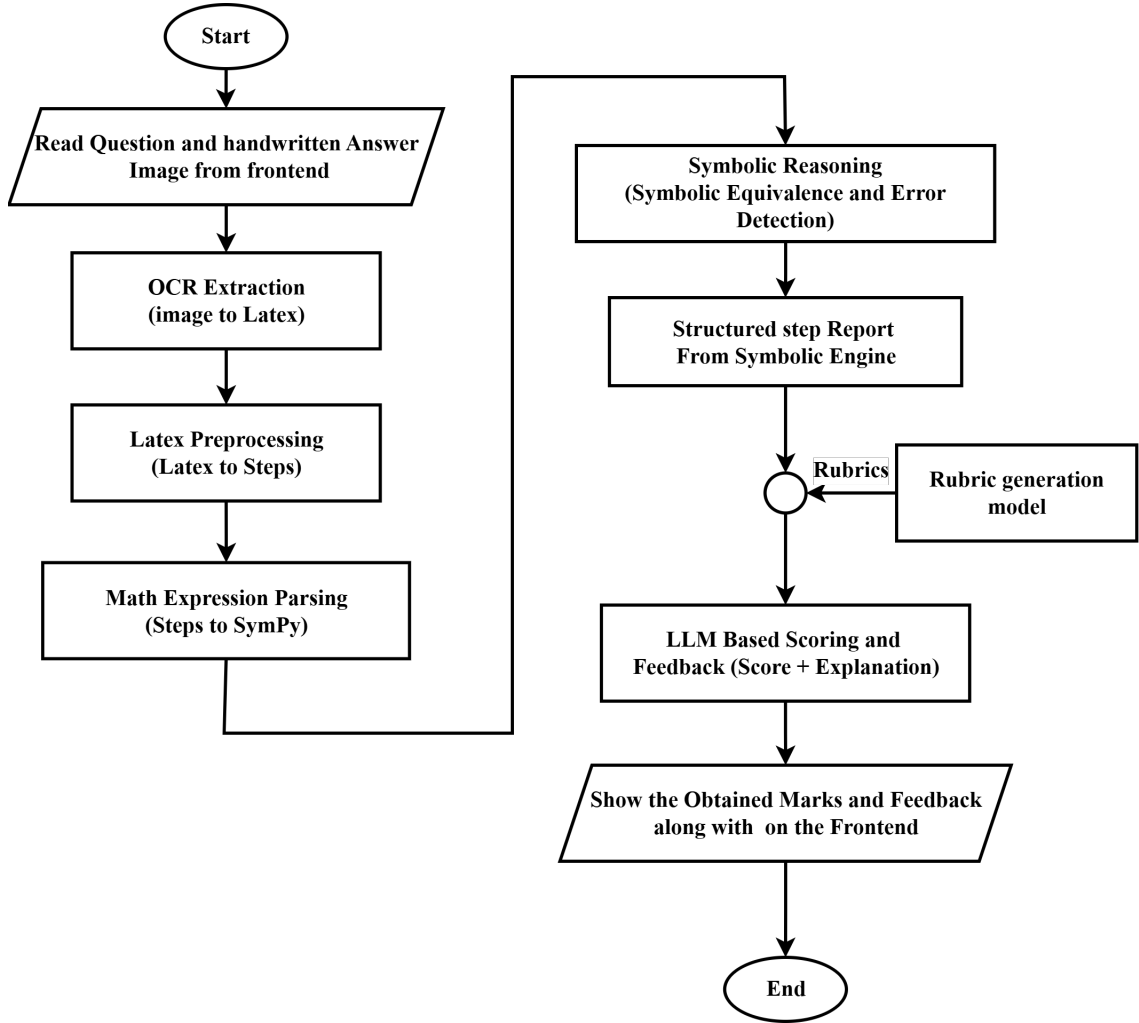


Figure 4-4: System Flow Diagram

4.3 Working Principle

4.3.1 Mathpix OCR

Mathpix OCR is a specialized optical character recognition (OCR) system designed to convert images of mathematical notation and scientific text into structured digital representations such as LaTeX, MathML, and Markdown. Unlike general-purpose OCR engines, Mathpix focuses on high-fidelity recognition of two-dimensional mathematical expressions, symbols, and layout-sensitive constructs using deep learning–based visual recognition.

4.3.1.1 Encoder–Decoder Architecture

Mathpix employs an encoder–decoder model:

- Encoder: CNN and Transformer that converts the image into a sequence or grid of feature embeddings
- Decoder: Sequence model (Transformer-based) that generates LaTeX tokens

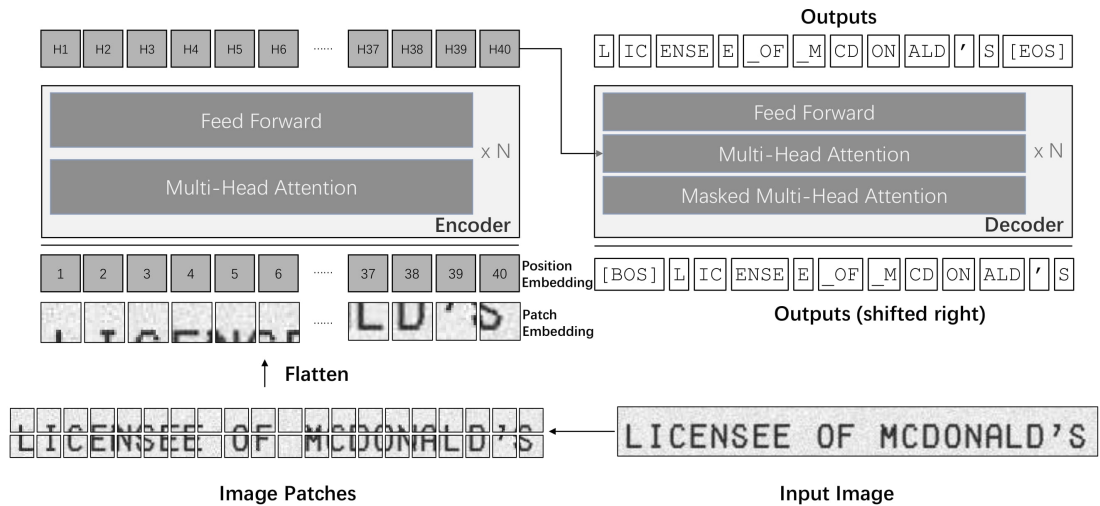


Figure 4-5: Block diagram of Mathpix OCR Architecture

Attention mechanisms allow the decoder to dynamically focus on relevant regions of the image while generating each token. For Nested expressions, Long equations and Multi-line environments attention mechanism is very suitable. Attention replaces heuristic symbol segmentation used in classical math OCR systems.

4.3.1.2 Output Formats

Mathpix can serialize recognized content into multiple formats:

- LaTeX: Primary and most expressive format
- MathML: For web rendering and accessibility
- Markdown: For documentation workflows
- Plain text: Limited mathematical fidelity

LaTeX remains the canonical intermediate representation due to its expressive power.

4.3.2 Data Collection

The very first requirement of any machine learning project is a collection of the relevant data from different sources. Data collection for automated grading systems includes a marking schema or marking rubrics as a reference in order to give marks for the students solutions. Marking rubrics contains a critical steps (checkpoints) that is contained with certain partial marks (reward points) of total marks. Those set of rubrics facilitates the examiner (In our case: Marking system) to check the multistep students solutions effectively, effortlessly and efficiently although there is a subjective bias if we use human evaluator. Hence this system certainly removes those subjective biases and evaluates objectively as per marking criteria defined by the marking rubrics.

For Basic Level Examination (BLE) and Secondary Education Examination (SEE) in Nepal, Curriculum Development Center (CDC) and National Examination Board (NEB) publishes the detailed marking schemes which are available by province (Koshi, Madhesh, Bagmati, Gandaki, Lumbini, Karnali, and Sudurpaschim). The sample marking rubrics that covers the Algebra Section from SEE model set 2081 (2025) Bagmati is shown in **Figure 4-6**.

Question

1. (a) सरल गर्नुहोस् (Simplify): (2)

$$\frac{p+q}{pq} - \frac{q+r}{qr} - \frac{r+p}{rp}$$

- (b) हल गर्नुहोस् (Solve): (3)

$$3^y + 3^{y-1} = 9\frac{1}{9}$$

Marking Rubric

Q.No.	Sub Q.	Steps / Answer	Marks
1	a	(i) $\frac{r(p+q) - p(q+r) - q(r+p)}{pqr}$	1
		(ii) $-\frac{2}{r}$	1
	b	(i) $9(3^y)^2 - 82(3^y) + 9 = 0$	1
		(ii) $(9 \cdot 3^y - 1)(3^y - 9) = 0$	1
		(iii) $y = -2, 2$	1

Figure 4-6: Original SEE Sample Question along with its Marking Rubrics published by NEB

We manually collected such questions along with its marking rubrics from various available sources which includes:

- i. Official websites of NEB, CDC
- ii. Textbooks from Class-7 to Class-10
- iii. Reference books (10 Sets, manuals)
- iv. Online mathematics materials
- v. Handwritten Solutions

Since, there was limited number of original marking rubrics available from above sources, we were able to collect only 312 samples. The samples were in the image/pdf format. As this quantity was not sufficient, we planned to augment and scale the dataset upto 2700 samples.

4.3.3 Data Preprocessing / Preparation

4.3.3.1 OCR-Based Text Extraction

Data preprocessing is a critical stage in the dataset preparation pipeline, as the raw image data was obtained after image collection. The images were passed through Optical Character Recognition (OCR) provided by Google's Gemini 3 model.

The OCR system was responsible for detecting and recognizing:

- Alphanumeric characters
- Mathematical symbols
- Structural elements such as fractions, powers, and parentheses

4.3.3.2 OCR Noise and Error Analysis

The initial OCR output often contained various forms of noise due to handwriting variations, image quality issues, and OCR misinterpretations. Common issues included:

- Incorrectly recognized mathematical symbols (e.g., '-' misread as '~', '1' as 'l')
- Broken expressions caused by improper line segmentation
- Random characters and meaningless strings introduced by background artifacts

4.3.3.3 Manual Verification and Content Filtering

The results were manually checked one by one through visual inspection as the results were not too high in number. Non-essential elements were removed which includes:

- Plain text explanations not contributing to the solution logic
- Page headers, footers, or annotations
- Diagrams, drawings, and unrelated symbols
- OCR-induced garbage strings

The two primary remaining content categories were formed as:

- Questions: Problem statements and its metadata (e.g., marks)
- Solutions: Step-by-step derivations corresponding to specific questions

4.3.3.4 Dataset Completion and Quality Control

In several cases, the dataset contained questions without corresponding solutions. To ensure dataset completeness, the solutions were added manually to the questions without the solutions. Other data samples that were found to be incomplete, corrupted, irrecoverable, or duplicated were excluded from the dataset. These included entries with missing question statements, partially extracted or inconsistent solution steps, severely distorted OCR outputs, repeated instances of identical or near-identical questions and solutions, and malformed mathematical expressions that could not be reliably corrected or validated.

Keeping such samples in the dataset would create redundancy, noise and bias that could compromise model generalization and evaluation accuracy. To maintain data quality, diversity and consistency, all such low-quality or redundant data entries were deliberately removed during the preprocessing stage.

4.3.3.5 Stepwise Mathematical Segmentation

The cleaned text was then segmented into logical mathematical units after noise removal. Solutions were broken into individual steps that corresponded to a student's stepwise problem solving process. It was important to define these segments to allow for partial marking and for a stepwise evaluation.

Segmentation was based on:

- Mathematical operators and equal signs
- Splits and boundaries of expression
- A logical sequence of algebraic transformations

Each solution was defined by an ordered sequence of atomic mathematical steps, not by a single monolithic expression.

4.3.3.6 LaTeX Transformation

Both questions and solutions were transformed into LaTeX format. It provides a structured, symbolic, and unambiguous mathematical representation. LaTeX provides a standardized way to encode mathematical expressions. LaTeX is suitable for the following:

- Formula parsing
- Stepwise comparison
- Equivalence checking
- Automate the evaluation process

Result from LaTeX transformation:

Input image files are converted into LaTeX, it is shown in Listing 1.

```
\item $\frac{1}{x - 1} - \frac{2}{x^2 - 1}$ m = 2 \\
$ = \frac{1}{x - 1} - \frac{2}{(x + 1)(x - 1)}$ m = 1 \\
$ = \frac{1}{x + 1}$ m = 1

\item $2^x + \frac{1}{2^x} = 4\frac{1}{4}$ m = 4 \\
$ = (2^x)^2 - \frac{17}{4}(2^x) + 1 = 0$ m = 1 \\
$ = 4(2^x)^2 - 17(2^x) + 4 = 0$ m = 1 \\
$ = (2^x - 4)(4 \cdot 2^x - 1) = 0$ m = 1 \\
$x = 2, -2$ m = 1

\item Simplify: $\log_a 36 + \frac{1}{2} \log_a 256 - 2 \log_a 48$ m = 2 \\
$ = \log_a 36 + \log_a 16 - \log_a 2304$ m = 1 \\
$ = \log_a (\frac{576}{2304})$ m = 0.5 \\
$ = -\log_a 4$ m = 0.5
```

Listing 1: Raw image to LaTeX Transformation

After the compilation of LaTeX it is appeared as follows:

1. Simplify: $\frac{1}{x-1} - \frac{2}{x^2-1}$ m = 2
 $= \frac{1}{x-1} - \frac{2}{(x+1)(x-1)}$ m = 1
 $= \frac{1}{x+1}$ m = 1
2. Solve: $2^x + \frac{1}{2^x} = 4\frac{1}{4}$ m = 4
 $= (2^x)^2 - \frac{17}{4}(2^x) + 1 = 0$ m = 1
 $= 4(2^x)^2 - 17(2^x) + 4 = 0$ m = 1
 $= (2^x - 4)(4 \cdot 2^x - 1) = 0$ m = 1
 $x = 2, -2$ m = 1
3. Simplify: $\log_a 36 + \frac{1}{2} \log_a 256 - 2 \log_a 48$ m = 2
 $= \log_a 36 + \log_a 16 - \log_a 2304$ m = 1
 $= \log_a \left(\frac{576}{2304}\right)$ m = 0.5
 $= -\log_a 4$ m = 0.5

Here, ‘m’ represents full marks for the question and partial marks for each solution steps.

4.3.3.7 Dataset Structuring and Annotation

The dataset was serialized into a structured JSON file. We picked JSON because it is hierarchical, readable and compatible with machine learning pipelines. For the sake of automated marking and finer evaluation, the dataset was also augmented with additional metadata for each question-answer pair. It is given below:

- Question type:

$$Topic = \begin{cases} \text{SIM,} & \text{Algebraic simplification} \\ \text{EQ,} & \text{Algebraic equation} \\ \text{LOG,} & \text{Logarithm and Indices} \end{cases} \quad (4-3)$$

- Difficulty level:

$$Difficulty = \begin{cases} -1, & \text{Easy} \\ 0, & \text{Medium} \\ 1, & \text{Hard} \end{cases} \quad (4-4)$$

Where:

$$\text{Difficulty Level} = \begin{cases} \text{Easy,} & \text{if total marks} = 1 \\ \text{Medium,} & \text{if total marks} = 2 \\ \text{Hard,} & \text{if total marks} > 2 \end{cases}$$

- Maximum marks allotted
- Stepwise marking scheme with normalized marks
- Acceptable equivalent forms for each solution step

4.3.3.8 Marks Normalization

The “marks” value is normalized using the min–max normalization technique, which transforms the marks range between 0 and 1 such that the sum of total marks equals 1.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (4-5)$$

Since $\min(x) = 0$,

$$x' = \frac{x}{\max_marks} \quad (4-6)$$

Where:

- x' : The normalized marks value (in the range $[0, 1]$)
- x : The original marks value
- $\sum x' = 1$

One sample of dataset is given below:

```
{
  "question": {
    "text": "Solve:  $5^x + 5^{-x} = 25 \times \frac{1}{25}$ ",
    "topic": "EQ",
    "difficulty": 1,
    "max_marks": 4
  },
  "rubric": [
    {
      "step_id": "Q37S1",
```

```

    "expected_forms": [
      "= 5^x + \\frac{1}{5^x} = \\frac{626}{25}"
    ],
    "weight": 0.25
  },
  {
    "step_id": "Q37S2",
    "expected_forms": [
      "= 25(5^x)^2 - 626(5^x) + 25 = 0"
    ],
    "weight": 0.25
  },
  {
    "step_id": "Q37S3",
    "expected_forms": [
      "= (5^x - 25)(25 \\cdot 5^x - 1) = 0"
    ],
    "weight": 0.25
  },
  {
    "step_id": "Q37S4",
    "expected_forms": [
      "x = 2, -2",
      "x = -2, 2"
    ],
    "weight": 0.25,
    "is_final": true
  }
]
}

```

More details about dataset can be seen in the Appendix Section D.

4.3.3.9 Data Augmentation

Out of 312 samples, after preprocessing 219 samples were finalized. We scale the data upto 2700 samples approximately using the various methods as:

- **Variable Substitution**

Systematically replaced variables (x, y, a, b) with different letters while preserving mathematical structure and relationships.

$$x^2 + 5x + 6 \rightarrow t^2 + 5t + 6, \quad y^2 + 5y + 6 \quad (4-7)$$

- **Coefficient Scaling**

Scaled numeric coefficients by factors of 2, 3, or 5, applied to arithmetic computation problems.

$$49^2 - 10^2 \rightarrow 98^2 - 20^2, \quad 147^2 - 30^2 \quad (4-8)$$

- **Structural Variations**

Reordered polynomial terms while maintaining equivalence.

$$x^2 + 5x + 6 \rightarrow 6 + 5x + x^2 \quad (4-9)$$

- **Difficulty Progression**

Created easier versions by using intermediate steps as new questions. Extracted steps from multi-step problems to generate simpler variants.

- **Factorization Templates**

Quadratic:

$$x^2 + bx + c = (x + p)(x + q), \quad \text{where } b = p + q, \quad c = pq \quad (4-10)$$

Difference of squares:

$$a^2 - b^2 = (a + b)(a - b) \quad (4-11)$$

Perfect squares:

$$a^2 \pm 2ab + b^2 = (a \pm b)^2 \quad (4-12)$$

Sum and difference of cubes:

$$a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2) \quad (4-13)$$

- **Exponent Law Templates**

Multiplication:

$$x^a \times x^b = x^{a+b} \quad (4-14)$$

Division:

$$\frac{x^a}{x^b} = x^{a-b} \quad (4-15)$$

Power rule:

$$(x^a)^b = x^{ab} \quad (4-16)$$

Zero exponent:

$$x^0 = 1 \quad (4-17)$$

• Fraction Templates

Common denominator:

$$\frac{a}{c} + \frac{b}{c} = \frac{a+b}{c} \quad (4-18)$$

Least common denominator (LCD) problems:

$$\frac{1}{x+a} \pm \frac{1}{x-a} \quad (4-19)$$

Algebraic cancellation:

$$\frac{x^2 - a^2}{x + a} = x - a \quad (4-20)$$

4.3.4 Exploratory Data Analysis

Rubric dataset consists of 2797 samples (Question and Rubric pairs). The sample distribution of the questions with respect to the topics (SIM: Simplify, LOG: Logarithm, EQ: Equation) is displayed in **Figure 4-7**.

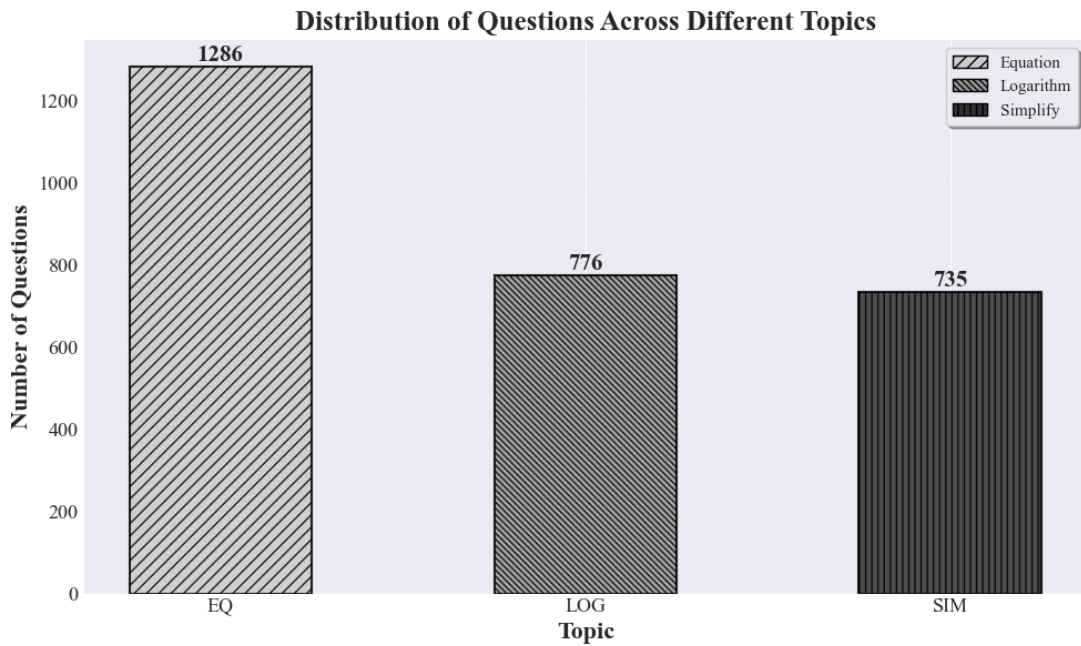


Figure 4-7: Distribution of Dataset Across Different Topics

Furthermore, the questions are divided into three sub topics for each topics based on the marks and difficulty level (-1: Easy, 0: Medium, 1: Hard) as shown in **Figure 4-8**.

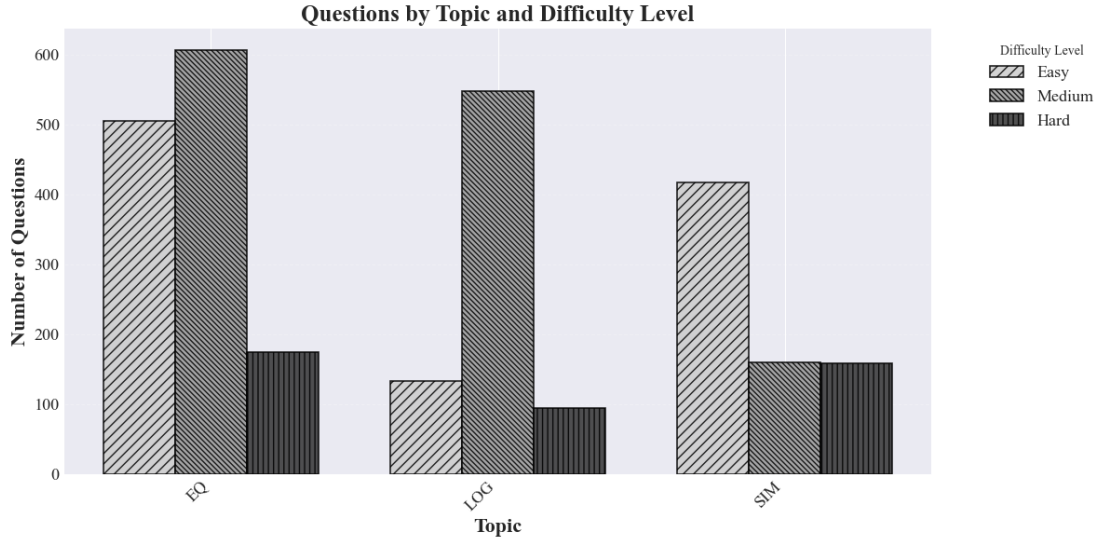


Figure 4-8: Distribution of Dataset Across Difficulty levels per Topics

4.3.5 DeepSeekMath-7B Model

In this system’s case, DeepSeek-Math 7B is used to perform two core tasks: first, generating a rubric of valid solution steps for a given mathematical problem; and second, evaluating a student’s step-by-step solution against that rubric to assign marks and generate feedback.

The grading task is framed as a structured text generation problem. The input prompt includes the math question, the OCR-processed student answer (converted to LaTeX or structured math), and optionally a rubric-like guide of expected steps. We format this using clear instruction-style prompts (e.g., “Grade the following solution step-by-step:”) followed by the complete context. The model is expected to generate an output that evaluates each step, indicates if it is correct, offers feedback and concludes with a final score summary. This is consistent with the multi-step reasoning capabilities of DeepSeek-Math.

We plan to fine-tune DeepSeek-Math 7B on a custom dataset of input-output pairs for rubric generation and stepwise grading. For the first task, the model learns to decompose problems into logical rubric steps with the marks given. Its second task is to compare student solutions to rubrics and generate graded, human-readable feedback.

4.3.5.1 Model Overview

DeepSeekMath-7B, a large language model for mathematical reasoning tasks developed by DeepSeek-AI [16]. We design our model based on the DeepSeek-Coder architecture to support multi-step reasoning, symbolic manipulation, and formal proof generation.

DeepSeekMath was launched in February 2024 and provides models of different sizes (1.3B, 7B and larger variants). The 7B parameter model provides the best trade-off between computational efficiency and reasoning capabilities, allowing deployment on consumer-grade hardware and cloud platforms such as Google Colab. 7B model was selected for our work due to its demonstrated ability to perform algebraic reasoning, good performance on math benchmarks (GSM8K, MATH) [17, 18], and ease of fine-tuning with limited computational resources.

Unlike general-purpose language models, DeepSeekMath-7B is pre-trained on a curated mathematical corpus. The corpus includes the following items:

- textbooks and educational materials of mathematics
- Research papers from arXiv (both math and physics)
- Competitive mathematics problem and solution pairs
- Formal proofs and derivations of mathematics
- Code repositories which contains mathematical algorithms

For automated marking systems this kind of specialized training is must. It makes accurate interpretation of mathematical notation and logical reasoning chains.

4.3.5.2 Architecture Details

DeepSeekMath-7B-Instruct is based a decoder-only transformer architecture. The architecture is fundamentally built upon the architectural principles introduced by Vaswani et al. [8] in their seminal “Attention is All You Need” paper.

The transformer is a standard architecture for building large language models. A transformer has three major components:

- i. Input encoding component
- ii. Transformer blocks
- iii. Language modeling head

Figure 4-9 shows the block diagram of a transformer architecture for language modeling.

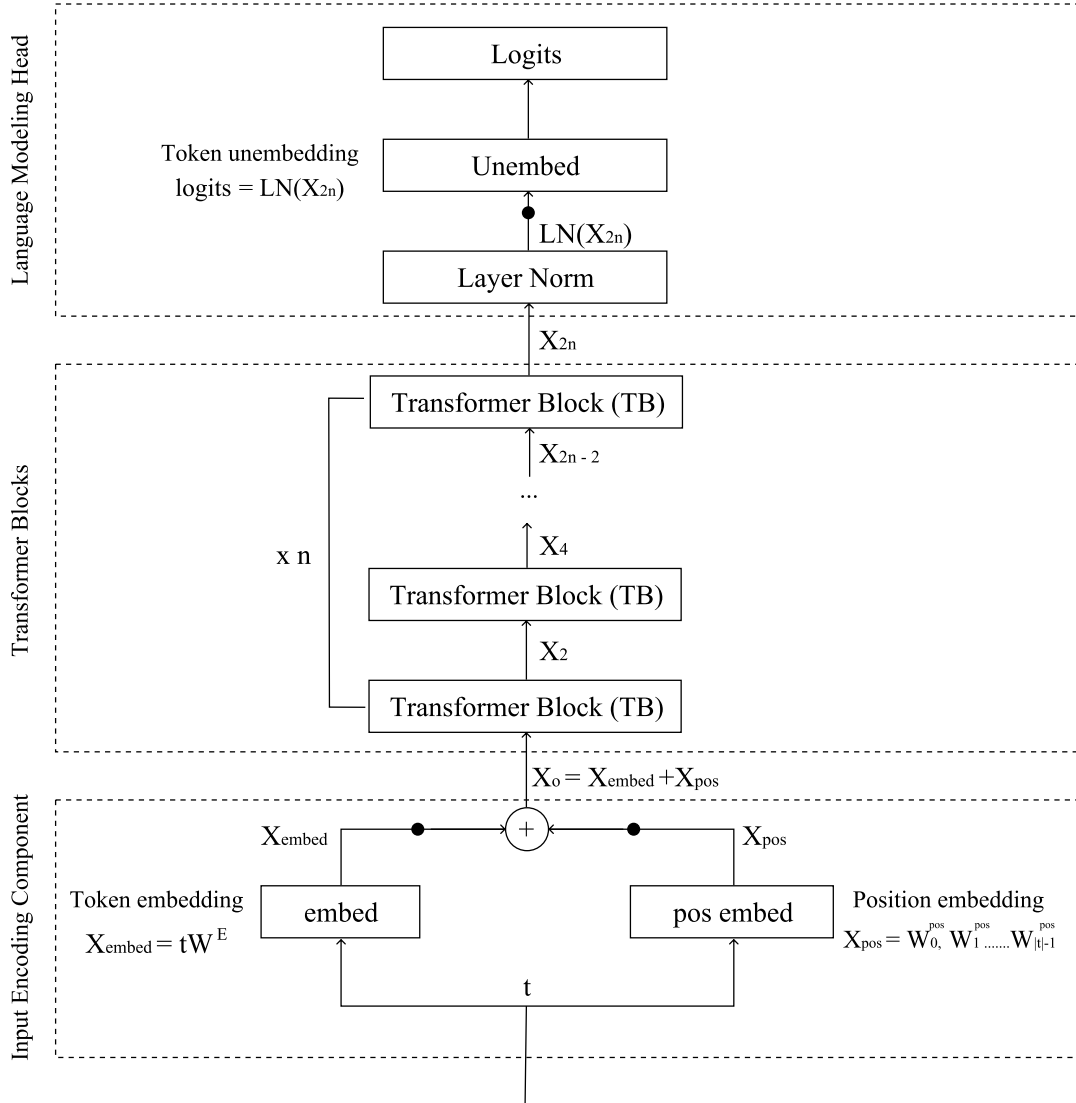


Figure 4-9: Block Diagram of Transformer Architecture

DeepSeekMath-7B-Instruct incorporates several modern innovations and optimizations that distinguish it from vanilla transformers.

i. Transformer Block Structure

The model consists of 30 transformer decoder layers stacked sequentially. Each layer as shown in Figure 4-10 contains:

- Multi-Head Self-Attention (32 heads)
- Layer Normalization (Pre-LN configuration)
- Feed-Forward Network (MLP)

- Linear: $d_{\text{model}} \rightarrow 4 \times d_{\text{model}}$ (expansion)
- Activation: SwiGLU
- Linear: $4 \times d_{\text{model}} \rightarrow d_{\text{model}}$ (projection)
- Residual Connection

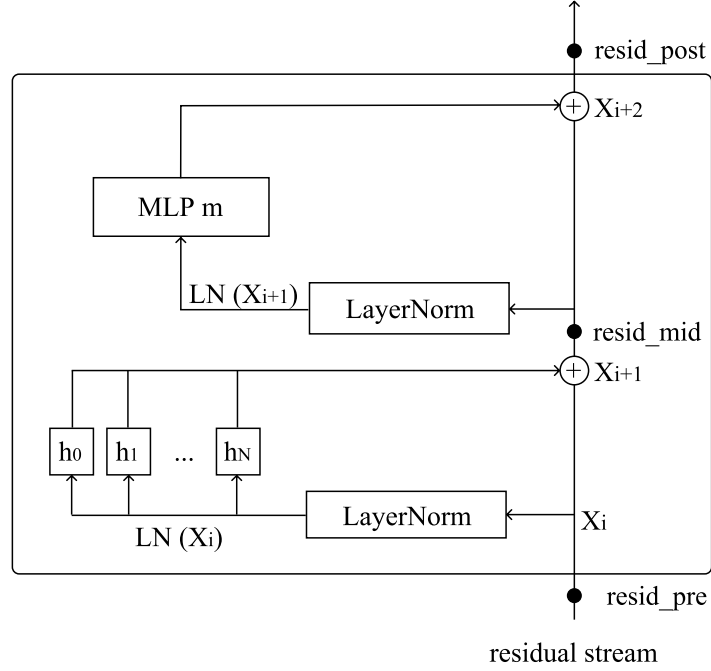


Figure 4-10: Transformer Block

In a transformer, layer by layer, richer and richer contextualized representations of the meanings of input tokens are built using the attention mechanism. At the highest transformer blocks, the residual stream is usually representing the following token.

ii. Model Dimensions

Table 4-1 shows the dimensions of DeepSeekMath model.

iii. Positional Encoding

The positional embedding can be thought of as a lookup table mapping the index of the position of each token to a residual stream vector. This helps transformers to have access to positional information so they know two tokens are next to each other (and hence probably relevant to each other). An input embedding that captures positional information is produced by adding the token embedding for each input to its corresponding positional embedding. The individual token and position embedding are both of shape $[1 \times d]$, so their sum is also $[1 \times d]$. Unlike the original transformer which used sinusoidal positional encodings, DeepSeek-

Table 4-1: DeepSeekMath-7B model dimensions

Parameter	Value
Hidden size (d_{model})	4096
Number of attention heads	32
Attention head dimension	128
FFN intermediate size	11,008
Vocabulary size	102,400
Maximum sequence length	4096 tokens
Number of layers	30

Math employs Rotary Position Embeddings (RoPE) [19]. RoPE encodes positional information by rotating the query and key representations in the attention mechanism:

$$q_m = f(x_m, m) = R_m^d W_q x_m \quad (4-21)$$

$$k_n = f(x_n, n) = R_n^d W_k x_n \quad (4-22)$$

where R_m^d is a rotation matrix dependent on position m and dimension d . This approach offers several advantages:

- Better length extrapolation (can handle sequences longer than training length)
- Improved relative position modeling
- More efficient than absolute positional encodings

iv. Attention Layer and its Mechanism

Each attention layer in a transformer moves information between pairs of embeddings. Attention layers determine which pairs should interact and what information should flow between them. Thus, attention layers can be seen as iteratively transforming the embedding vectors to build a rich contextualized representations of the meaning of input tokens. Attention layers are made up of a number of heads - each with their own parameters, attention pattern, and information on how to copy things from source to destination as shown in figure 4-11. These heads act independently and additively.

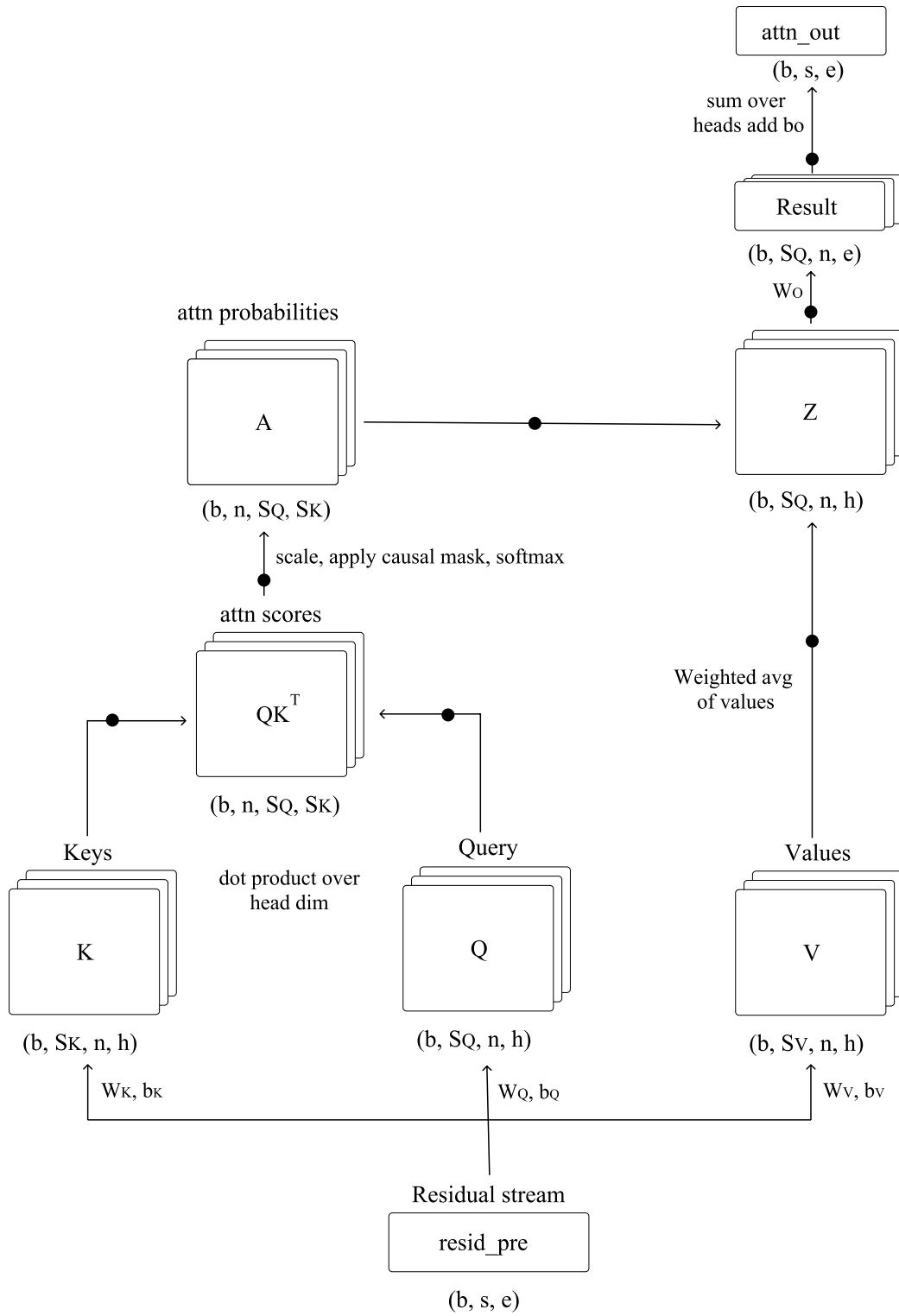


Figure 4-11: Attention Heads

In figure 4-11,

b = batch size

s = sequence length

e = embedding dimension

n = number of heads

h = head size (also called d_{head} , or d_k)

W_K, W_Q, W_V are projection matrices of shape (n, e, h)

b_K, b_Q, b_V are biases of shape (n, h)

W_O is a projection matrix of shape (n, h, e)

b_O is a bias of shape (e)

Assuming that $\mathbf{x}$ represents the contextual embedding vector at `resid_prein` in figure 4-11, the attention scores is given by,

$$\text{attn scores} = QK^T = \mathbf{x}W_QW_K^T\mathbf{x}^T \quad (4-23)$$

The attention scores are scaled and masked, which is then used for computing the attention probabilities,

$$A = \text{softmax}\left(\frac{\mathbf{x}W_QW_K^T\mathbf{x}^T}{\sqrt{d_{head}}}\right) \quad (4-24)$$

For each key position, we take a weighted average of value vectors from each query position, in accordance with how much attention destination pays to source,

$$\mathbf{z} = A\mathbf{x}W_V \quad (4-25)$$

OR

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4-26)$$

A final linear transformation is applied to $\mathbf{z}$ by the projection matrix W_O mapping the vectors into the right size to be added to the residual stream

$$\begin{aligned} \text{result} &= \mathbf{z}W_O \\ &= A\mathbf{x}W_VW_O \end{aligned} \quad (4-27)$$

Finally, the heads are summed over to gain the output of the attention layer as

shown in figure 4-11.

For computational efficiency, DeepSeekMath implements Grouped-Query Attention (GQA), where:

- Query heads: 32
- Key-Value heads: 8
- Grouping ratio: 4:1

This reduces the memory footprint during inference while maintaining model quality, as keys and values are shared across multiple query heads.

v. Residual Stream

The residual stream is the sum of all previous outputs of layers of the model and also the input to each new layer in the model. The residual stream is fundamental as it is the central object of the transformers, as it is how the model:

- remembers information
- moves information between layers for composition
- stores the information that attention moves between positions.

vi. Feed-Forward Network

Once attention has moved relevant information to a single position in the residual stream, Feed forward network can actually do computation, reasoning, lookup information, etc.

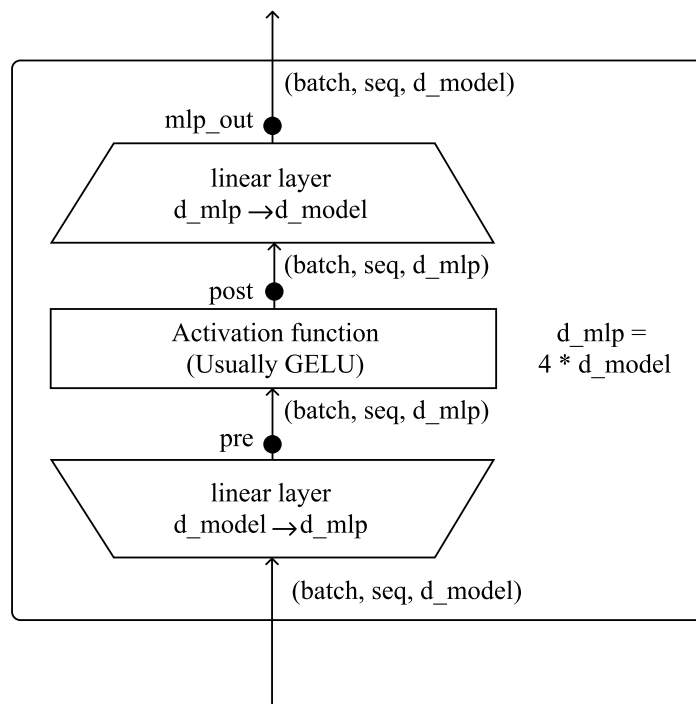


Figure 4-12: Multi-layer Perceptron

Feed forward layers are just a standard neural network, with a singular hidden layer and a non-linear activation function. The hidden dimension is normally, $d_{mlp} = 4 * d_{model}$ as shown in Figure 4-12.

Each transformer layer includes a position-wise feed-forward network using the SwiGLU activation function [20]:

$$\text{FFN}(x) = \text{Linear}_2(\text{SwiGLU}(\text{Linear}_1(x))) \quad (4-28)$$

where:

$$\text{SwiGLU}(x) = \text{Swish}(xW_1 + b_1) \otimes (xW_2 + b_2) \quad (4-29)$$

SwiGLU has been shown to outperform traditional ReLU and GELU activations in language modeling tasks, providing:

- Smoother gradient flow
- Better handling of negative values
- Improved training stability

vii. Layer Normalization

LayerNorm is a simple normalization function applied at the start of each layer. It converts each input vector to have zero mean and unit variance, and then applies an element-wise scaling and translation; This abstraction is displayed in Figure 4-13.

The model uses Pre-Layer Normalization (Pre-LN) configuration with RMSNorm:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \cdot g \quad (4-30)$$

where:

$$\text{RMS}(x) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \quad (4-31)$$

Pre-LN places normalization before the attention and FFN sub-layers rather than after, which:

- Stabilizes training of deep networks
- Reduces gradient explosion/vanishing
- Eliminates the need for learning rate warmup in some cases

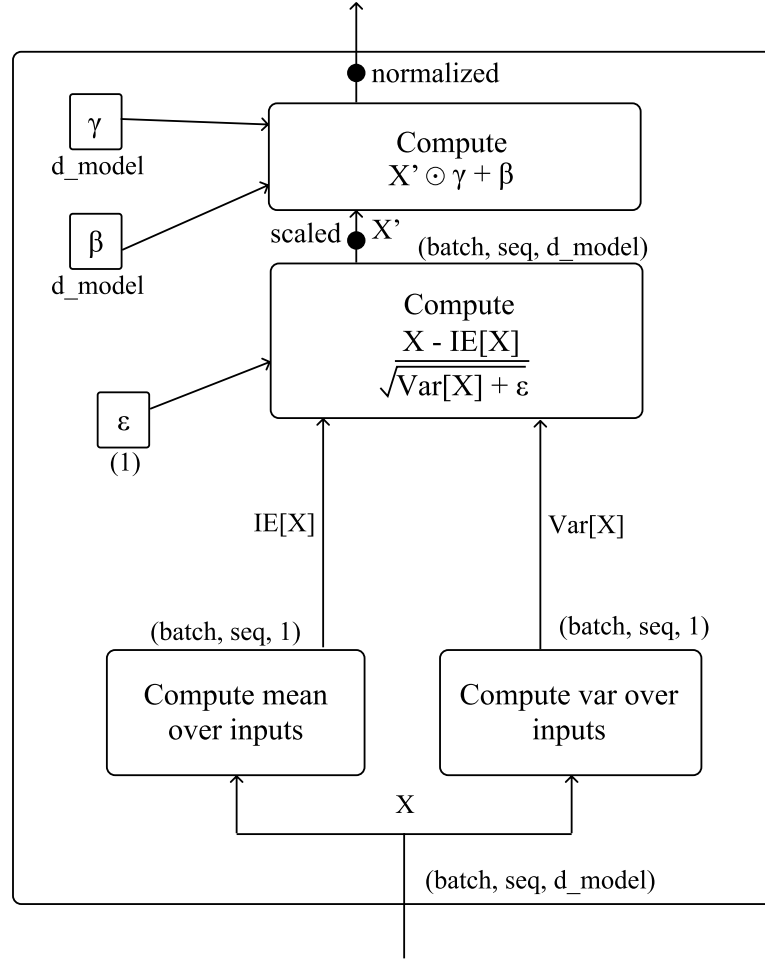


Figure 4-13: LayerNorm

viii. Language Modeling Head

Here, the word head refers to the additional neural circuitry that is added on top of the basic transformer architecture when we apply pretrained transformer models to various tasks. The language model head is the circuitry we need to do language modeling. The task of the language modeling head is to take the output of the final transformer layer from the last token N and use it to predict the upcoming word at position $N + 1$.

The language modeling head consists of a linear layer and a softmax layer.

(a) Unembedding Layer

The unembedding layer is a linear layer which projects the output h_N^L (the output token embedding at position N from the final transformer block L) to the logit vector that will have a single score for each of the $|V|$ possible words in the vocabulary V . The logit vector, $\mathbf{u}$, has a dimensionality of $1 \times |V|$.

$$\mathbf{u} = h_N^L W_U \quad (4-32)$$

(b) **Softmax Layer**

The logits $\mathbf{u} \in \mathbb{R}^V$ from the unembedding layer are converted into a probability distribution $\mathbf{y}$ over the vocabulary using the softmax function:

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \quad (4-33)$$

where each element y_i is given by

$$y_i = \frac{\exp(u_i)}{\sum_{j=1}^V \exp(u_j)}, \quad i = 1, 2, \dots, V \quad (4-34)$$

Here, V denotes the vocabulary size. The softmax function ensures that $0 \leq y_i \leq 1$ and $\sum_{i=1}^V y_i = 1$, making $\mathbf{y}$ a valid probability distribution over the vocabulary.

To generate text, a word is sampled from the probabilities $\mathbf{y}$ obtained from the Softmax layer. The sampling can be done in the following ways:

- (a) Greedy Decoding
- (b) Random Sampling
- (c) Top-k Sampling
- (d) Top-p (Nucleus) Sampling
- (e) Temperature Sampling

4.3.5.3 Tokenization

DeepSeekMath uses a Sentence Piece tokenizer with a vocabulary of 102,400 tokens. The vocabulary corpus was specifically designed to efficiently encode:

- Mathematical symbols ($\Sigma, f, \partial, \alpha, \beta$, etc.)
- LaTeX commands ($\backslash\text{frac}$, $\backslash\text{sqrt}$, $\backslash\text{int}$, etc.)
- Numbers and numeric expressions
- Common mathematical terms
- Programming constructs (for code-based solutions)

It learns a fixed-size vocabulary of subword units by optimizing the likelihood of the training data, allowing frequently occurring patterns to be represented as single tokens, while rare or complex expressions are decomposed into smaller subword components.

This specialised vocabulary cuts down the token count of mathematical expressions by about 30% compared to general-purpose tokenisers. This results in:

- More effective use of context
- Lower inference latency
- Shorter token sequences, making attention more efficient
- Improved preservation of mathematical structure

SentencePiece, in contrast to word-level tokenisation, effectively handles the following:

- Mathematical expressions without spaces, continuous
- LaTeX nested symbols and commands
- Numbers and equations of different length
- Noise due to partial symbol recognition and OCR

Consider the following mathematical expression:

$$\frac{d}{dx}(x^2) \quad (4-35)$$

Word level tokenisers tend to split above expression into individual characters or arbitrary sub strings which do not carry any mathematical meaning on their own. SentencePiece splits the same expression into subword units that correspond to recognisable math- So the structure of the expression is preserved in the token sequence and not gets fragmented.

4.3.5.4 Training Methodology

Parameter-Efficient Fine-Tuning (PEFT)

The Parameter Efficient Fine-Tuning technique is best suited for fewer computational constraints.

i. LoRA (Low-Rank Adaptation)

When fine-tuning a pre-trained large-scale model, retaining all model parameters becomes less feasible. LoRA introduces trainable low-rank decomposition matrices into each layer while keeping the pre-trained weights frozen [21]. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA represents the weight update as:

$$W = W_0 + \Delta W = W_0 + BA \quad (4-36)$$

where:

- $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable matrices
- $r \ll \min(d, k)$ is the rank hyperparameter
- W_0 remains frozen during training

The forward pass becomes:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (4-37)$$

During full fine-tuning, the model is initialized to pre-trained weights Φ_0 and updated to $\Phi_0 + \Delta\Phi$ by repeatedly following the gradient to maximize the conditional language modeling objective:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t | x, y_{<t})) \quad (4-38)$$

One of the main drawbacks for full fine-tuning is that for *each* downstream task, we learn a different set of parameters $\Delta\Phi$ whose dimension $|\Delta\Phi|$ equals $|\Phi_0|$. Thus, if the pre-trained model is large, storing and deploying many independent instances of fine-tuned models can be challenging, if at all feasible.

LoRA adopts a more parameter-efficient approach, where the task-specific parameter increment $\Delta\Phi = \Delta\Phi(\Theta)$ is further encoded by a much smaller-sized set of parameters Θ with $|\Theta| \ll |\Phi_0|$. The task of finding $\Delta\Phi$ thus becomes optimizing over Θ :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})) \quad (4-39)$$

In the subsequent sections, we propose to use a low-rank representation to encode $\Delta\Phi$ that is both compute- and memory-efficient.

Benefits:

- Reduces trainable parameters by $> 99\%$ ($7B \rightarrow 4.7M$ parameters)
- No inference latency overhead (can merge BA into W_0)
- Maintains model quality while drastically reducing memory

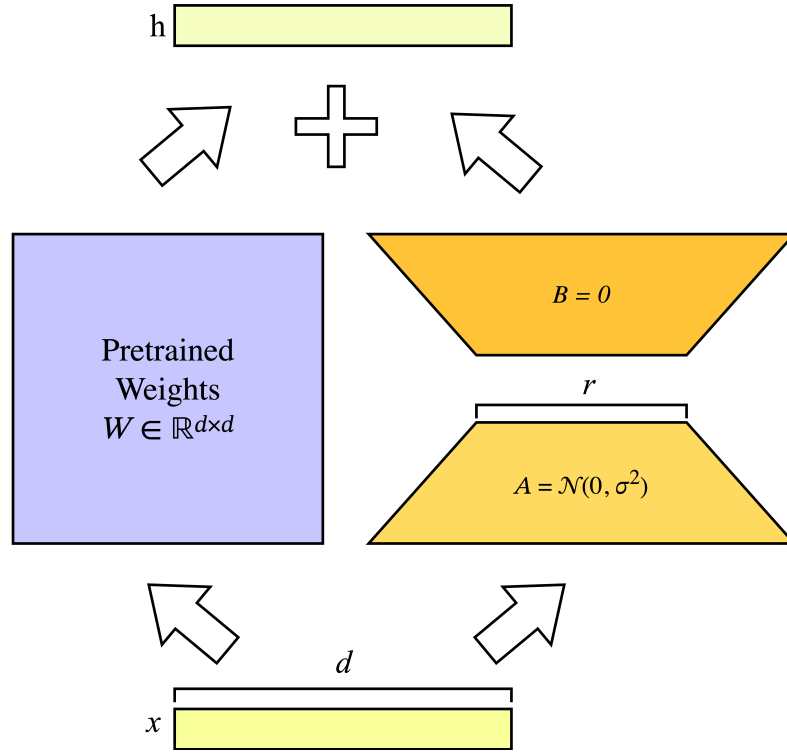


Figure 4-14: Block Diagram of LoRA reparametrization

ii. Quantization (QLoRA)

QLoRA extends LoRA by quantizing the frozen pre-trained weights to 4-bit precision, further reducing memory usage. The quantization process:

- (a) **NF4 (Normal Float 4-bit)**: Uses a theoretically optimal quantization scheme for normally distributed weights:

$$Q(w) = \text{quantize}_{\text{NF4}}(w/s) \quad (4-40)$$

where s is a learned scaling factor per block of weights.

- (b) **Double Quantization**: Quantizes the quantization constants themselves:

$$Q(s) = \text{quantize}_{\text{FP8}}(s) \quad (4-41)$$

- (c) **Paged Optimizers**: Uses NVIDIA Unified Memory to automatically handle memory spikes during training.

4.3.5.5 Mathematical Reasoning Capabilities

DeepSeekMath-7B-Instruct demonstrates several key capabilities relevant to our grading application:

Step-by-Step Reasoning

The model can decompose complex problems into logical steps:

```
Problem: Solve  $2x + 5 = 13$ 

Model Output:
Step 1: Start with the equation  $2x + 5 = 13$ 
Step 2: Subtract 5 from both sides:  $2x = 8$ 
Step 3: Divide both sides by 2:  $x = 4$ 
```

Listing 2: Example of step-by-step reasoning

Error Detection

The model can identify common mistakes in student solutions:

- Arithmetic errors
- Sign errors (incorrectly handling positive/negative)
- Invalid algebraic transformations
- Missing steps or logical jumps

Multiple Solution Paths

The model recognizes that mathematical problems can be solved through different valid approaches, as shown in Listing below:

```
Problem:  $x^2 - 4 = 0$ 

Method 1: Factoring  $\rightarrow (x-2)(x+2) = 0$ 
Method 2: Square root  $\rightarrow x^2 = 4 \rightarrow x = \pm 2$ 
Method 3: Quadratic formula
```

Listing 3: Multiple Solution Paths

4.3.5.6 Chain-of-Thought Reasoning

Chain-of-Thought (CoT) prompting [22] is a technique that elicits step-by-step reasoning from language models by including intermediate reasoning steps in the prompt. CoT has improved performance in complex reasoning tasks particularly in mathematical problem-solving.

Chain-of-Thought reasoning is helpful for Rubrics Generation and Step wise evaluation as described below.

Standard Prompting:

```
Q: Solve for x:  $2x + 5 = 13$   
A:  $x = 4$ 
```

Listing 4: Example of Standard Prompting

Chain-of-Thought Prompting:

```
Q: Solve for x:  $2x + 5 = 13$   
A: Let me solve this step-by-step:  
Step 1: Start with the equation:  $2x + 5 = 13$   
Step 2: Subtract 5 from both sides:  $2x = 8$   
Step 3: Divide both sides by 2:  $x = 4$   
Therefore,  $x = 4$ 
```

Listing 5: Example of Chain-of-Thought Prompting

Rubric Generation

In order to generate appropriate set of rubrics the model needs reasoning on the solution processes. The example of such rubrics generation is given below:

```
Input: "Create a rubric for: Solve  $x^2 - 5x + 6 = 0$ "  
Model's Internal Reasoning (CoT):  
- This is a quadratic equation  
- Can be solved by factoring or the quadratic formula  
- Key steps: factor, apply the zero product property, solve  
- Should allocate marks for each step-  
Output: Structured rubric with step-wise marking
```

Listing 6: Example of Rubric Generation

Step-by-Step Evaluation

During the grading process, the model is expected to:

- i. Analyze the intended solution approach
- ii. Compare each step of the student's solution with the marking rubric
- iii. Detect mistakes or incorrect reasoning in the solution

iv. Assign appropriate partial marks based on the work shown

```
Student Step 2: 2x = 18 (Incorrect: should be 8)
Model's CoT:
- Expected: 2x = 13 - 5 = 8
- Student has: 2x = 18
- Error: Arithmetic mistake (18 not equal to 8)
- Impact: Subsequent steps will be wrong
- Decision: Award 0 marks, note error type
```

Listing 7: Example of Step-by-Step Evaluation

4.3.6 Symbolic Reasoning Engine: SymPy Core Architecture

In our automated grading pipeline, prior to DeepSeekMath-based semantic evaluation, we employ SymPy as a deterministic symbolic reasoning engine to perform mathematically objective verification of student solutions. SymPy is a pure-Python computer algebra system (CAS) designed around exact symbolic computation rather than numerical approximation [5, 23]. Its architecture is particularly well-suited for educational auto-grading tasks, where correctness must be judged independently of expression form.

4.3.6.1 Core Design and Expression Representation

At the heart of SymPy lies a tree-based symbolic expression model. Every mathematical object (symbols, numbers, operators, functions, or equations) is represented as an immutable expression tree composed of instances of the base class *sympy.core.basic.Basic*. For example, an algebraic expression such as

$$(x+1)^2 \tag{4-42}$$

is internally represented as a hierarchical tree consisting of *Pow*, *Add*, *Mul*, and *Symbol* nodes.

For the expression: `expr = x*y + 2`. Using SymPy's internal representation, `srepr(expr)` returns

```
Add(Mul(Symbol('x'), Symbol('y')), Integer(2)).
```

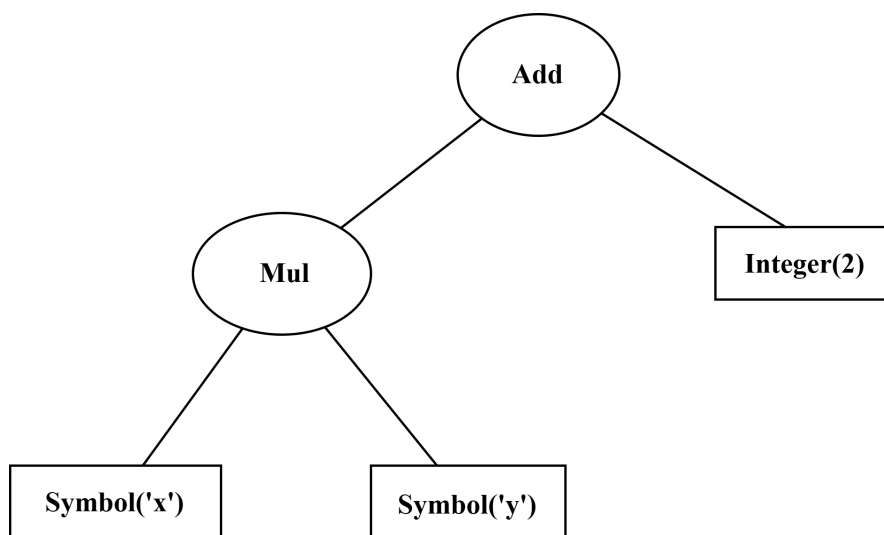


Figure 4-15: Parse tree for `expr = x*y + 2`

Figure 4-15 for the above expression shows that the top-level (parent) node of the expression tree is an *Add* object, corresponding to addition. Its two child nodes are a *Mul* object representing $x \cdot y$, and an *Integer* object representing the constant 2.

Descending further in the tree, the *Mul* node has two children, `Symbol('x')` and `Symbol('y')`. Both symbols and numeric constants are *leaf nodes*, which is reflected by their empty argument lists:

```
expr.args[0].args = ()
```

indicating that no further decomposition is possible. This hierarchical structure precisely encodes the syntactic and semantic form of mathematical expressions, enabling SymPy to apply transformations, simplifications, and equivalence checks through recursive tree traversal.

4.3.6.2 Parsing and Canonicalization

After OCR converts handwritten student solutions into $\text{\LaTeX}$, SymPy's *latex* parser translates each expression into this internal symbolic form. For instance,

```
sympy.parsing.latex.parse_latex(r"\frac{1+\sqrt{a}}{b}\{b\}")
```

is converted into the symbolic expression $\frac{1+\sqrt{a}}{b}$. Once parsed, SymPy applies canonicalization rules such as sorting commutative operands and normalizing algebraic forms so that mathematically equivalent expressions map to comparable internal representations, even if written differently by students.

4.3.6.3 Rule-Based Transformation and Simplification

SymPy checks algebraic equivalence by reducing expressions to a canonical form, not by testing numerical values. Functions like *expand*, *factor*, *simplify*, and *cancel* work by walking the expression tree and applying transformation rules from symbolic algebra. Two expressions are considered equal when their difference simplifies to zero:

$$\text{simplify}(\text{expression1} - \text{expression2}) = 0 \quad (4-43)$$

This is what makes the grading flexible. A student who writes $(x^2 - 1)$ instead of $(x - 1)(x + 1)$ is not wrong as both forms reduce to the same canonical expression and the equivalence check passes. No pattern matching is involved. So the system does not reject valid answers just because they look different from the model solution.

4.3.6.4 Equation Solving and Logical Verification

For problems containing equations, SymPy provides exact solvers that operate symbolically. This makes SymPy powerful tool as it treats expressions as symbols not numbers. For example,

$$\text{solve}(x^{**2} - 1, x) = [-1, 1] \quad (4-44)$$

verifies the roots of $x^2 - 1 = 0$.

Such solvers helps to validate intermediate as wells as final results against mathematically valid solution sets. This ensures correctness even when students follow novel but valid solution paths.

4.3.6.5 Stepwise Validation in Auto-Grading

The reason why SymPy expressions are useful for determining the validity of student steps is that they are immutable and structurally comparable. Symbolically comparing subsequent expressions would enable the detection of any illegal mathematical steps taken by the student (such as incorrect canceling out). The errors will be marked in a deterministic manner with messages such as “Step 3 algebraically incorrect”.

4.3.6.6 Integration with DeepSeekMath

The outputs from symbolic reasoning engine is used as input to the DeepSeekMath. While SymPy provides exact, rule-based correctness judgments, DeepSeekMath evaluates reasoning quality, partial credit, and explanatory coherence. Flags generated by SymPy, such as incorrect intermediate steps or equivalent final answers, can be incorporated into DeepSeekMath prompts or used to cross-validate its rubric-based assess-

ments. Overall, SymPy acts as the formal mathematical backbone of the marking system, ensuring objective correctness, while DeepSeekMath provides higher interpretability and feedback.

4.3.7 Optimization

Optimization is the process of updating the parameters of a model to minimize the loss function.

4.3.7.1 Momentum

Momentum is a method that can be used to accelerate SGD in relevant direction. It does so by moving faster in directions that had good gradients previously and slowing down in directions where gradient has suddenly changed.

It does so by adding a fraction of the past gradient to the current gradient,

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_{t-1} \quad (4-45)$$

where,

- $\mathbf{m}_t$ is the momentum,
- β is a fraction in the range $[0, 1]$, which controls the influence of past gradients in the current update, and
- $\mathbf{g}_{t-1}$ is the gradient at time step $t - 1$.

Then the parameters θ is be updated as,

$$\theta_t = \theta_{t-1} + \eta_t \mathbf{m}_t \quad (4-46)$$

where, η_t is the learning rate.

4.3.7.2 Adagrad

Adagrad [24] is an algorithm which adapts the learning rate to the parameters. At every time step t , it uses a different learning rate for every parameter θ , and its update rule is given by,

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{\sqrt{\mathbf{s}_t} + \epsilon} \mathbf{g}_t \quad (4-47)$$

where,

$$\mathbf{s}_t = \sum_{\tau=1}^t \mathbf{g}_\tau^2 \quad (4-48)$$

sum of the squared gradients, and ε is a small term added for numerical stability.

4.3.7.3 RMSProp

RMSProp [25] is an alternative to Adagrad which uses exponentially weighted moving average of the past squared gradients,

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \mathbf{g}_{t-1}^2 \quad (4-49)$$

Its update rule is given by,

$$\theta_t = \theta_{t-1} - \frac{\eta_t}{\sqrt{\mathbf{s}_t} + \varepsilon} \mathbf{g}_{t-1} \quad (4-50)$$

4.3.7.4 Adam

Adaptive Moment Estimation (Adam) [26] is another algorithm for computing the adaptive learning rates for each parameter. It combines RMSProps with momentum.

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_{t-1} \\ \mathbf{s}_t &= \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_{t-1}^2 \end{aligned} \quad (4-51)$$

Its update rule is given by,

$$\theta_t = \theta_{t-1} - \frac{\eta_t}{\sqrt{\mathbf{s}_t} + \varepsilon} \mathbf{m}_t \quad (4-52)$$

4.3.8 Activation Function

4.3.8.1 Gaussian Error Linear Unit

GELU (Gaussian Error Linear Unit) is an activation function commonly used in modern neural networks, especially transformers.

It is given by,

$$\text{GELU}(x) = x \cdot \Phi(x) \quad (4-53)$$

where, $\Phi(x)$ is the CDF of the standard normal distribution.

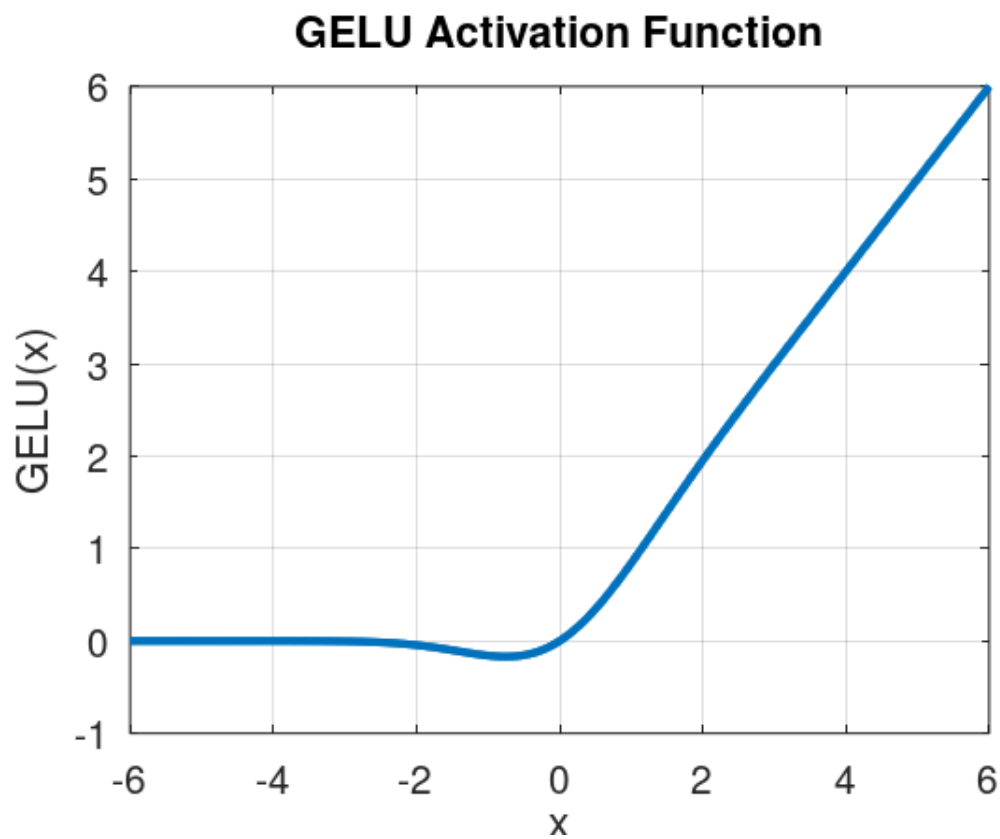


Figure 4-16: GELU Activation Function

It is smoother than ReLU and has better empirical performance in Transformers.

4.3.8.2 Swish Activation Function

The swish function, also known as Swish-1, is given as,

$$\text{SiLU}(x) = x \cdot \frac{1}{1 + e^{-x}} \quad (4-54)$$

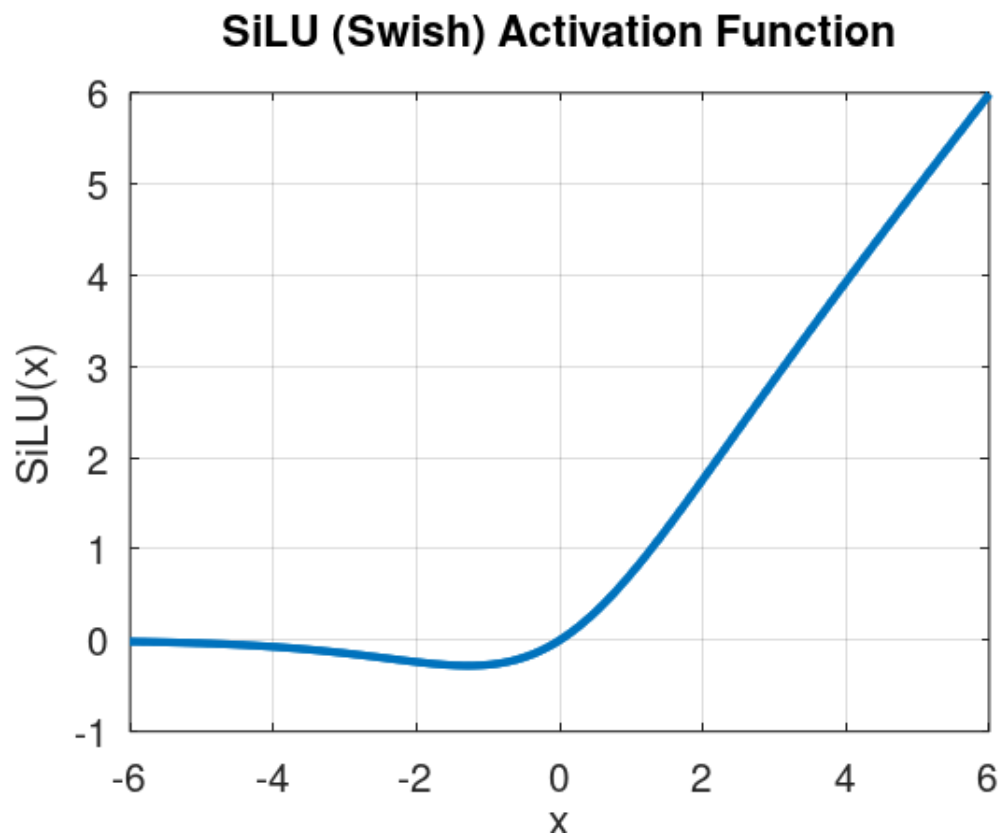


Figure 4-17: SiLU Activation Function

It is better than ReLU when it comes to the neuron dying problem but does not entirely mitigate it. It gives small negative outputs for negative inputs close to zero.

4.3.9 Evaluation Metrics

Evaluation metrics are tools that give us numbers to understand how well our models are working. Choosing the right metric is important because it highlights the most crucial aspects of performance for our particular task.

4.3.9.1 OCR Metrics

Character Error Rate (CER)

Measures the number of incorrect characters in the predicted expression compared to the ground truth.

$$CER = \frac{\text{Substitutions} + \text{Insertions} + \text{Deletions}}{\text{Total Characters}} \quad (4-4)$$

Lower CER values indicate better performance.

4.3.9.2 Answer Marking Metrics

Stepwise Weighted Accuracy (Per Method)

This metric evaluates how accurately a generated rubric reproduces the expected solution steps for a given method. Each method consists of a finite number of steps, and each step is assigned a predefined weight reflecting its importance within that method.

For a method m with N_m steps, let w_i be the weight of step i , s_i be the expected algebraic form, and $\hat{s}_i$ is the form generated by the model. We check correctness using algebraic equivalence not by exact string matching.

The stepwise weighted score for method m is:

$$\text{StepwiseScore}_m = \sum_{i=1}^{N_m} w_i \mathbf{1}[\hat{s}_i \equiv s_i] \quad (4-55)$$

Here the indicator function $\mathbf{1}[\hat{s}_i \equiv s_i]$ returns 1 if the predicted step is algebraically equivalent to the expected step, and 0 if not. This means a solution that gets only some steps right still earns partial credit proportional to the weights of those steps.

Final Answer Correctness

The final answer needs to be correct along with the intermediate steps to get the full marks. This metric checks whether the final answer in the rubric matches the canonical solution or not. It uses algebraic equivalence test not exact string comparison. If any multiple alternative forms that were mathematically equivalent to the final answer then that is also allowed and checked.

Let a be the canonical final answer and $\hat{a}$ the predicted one:

$$\text{FinalAnswerScore} = \begin{cases} 1, & \text{if } \hat{a} \equiv_{\text{canonical}} a, \\ 0, & \text{otherwise.} \end{cases} \quad (4-56)$$

Total Rubric Score (Per Method)

The total rubric score for a method m is the combination of all above three metrics:

$$\begin{aligned} \text{TotalScore}_m &= \text{StepwiseScore}_m + \text{MilestoneScore}_m \\ &\quad + \text{FinalAnswerScore} \end{aligned} \quad (4-57)$$

Depending on the requirements, this score is either normalized to the interval $[0, 1]$ or

scaled by the maximum marks assigned to the problem.

4.3.9.3 Model Evaluation Metrics

Parse Success Rate:

Before any grading can happen, the model's output has to be parseable into valid JSON with the expected rubric structure. This metric measures what percentage of outputs actually meet that requirement. An output that cannot be parsed is useless for subsequent stages.

For a test set of N examples, let $P_i = 1$ if output i parses successfully and $P_i = 0$ otherwise:

$$\text{Parse Success Rate} = \frac{1}{N} \sum_{i=1}^N P_i \times 100\% \quad (4-58)$$

Step Exact Match:

This is the primary metric for the quality of rubric generation. It checks whether the step identifiers in the generated rubric match those in the reference rubric position by position. Higher the value means the model is producing the right number of steps as well as in right order.

For rubric i with reference steps $R_i = \{r_1, r_2, \dots, r_{m_i}\}$ and generated steps $G_i = \{g_1, g_2, \dots, g_{n_i}\}$. The step match score is calculated as:

$$\text{Match}_i = \frac{1}{m_i} \sum_{j=1}^{\min(m_i, n_i)} \mathbf{1}[r_j.\text{step_id} = g_j.\text{step_id}] \quad (4-59)$$

where $\mathbf{1}[\cdot]$ is the indicator function returning 1 if the condition is true and 0 otherwise. The Step Exact Match across the test set is given by

$$\text{Step Exact Match} = \frac{1}{N} \sum_{i=1}^N \text{Match}_i \times 100\% \quad (4-60)$$

Number of Steps Match:

A rubric that has the right steps but the wrong count is still a bad rubric. This metric checks whether the model generates exactly the same number of steps as the reference. It helps to check whether the model understands the complexity of the problem.

For each rubric pair (R_i, G_i) :

$$S_i = \begin{cases} 1 & \text{if } |R_i| = |G_i| \\ 0 & \text{otherwise} \end{cases} \quad (4-61)$$

where:

$|R_i|$ denote the number of steps in reference rubrics, and

$|G_i|$ denote the number of steps in generated rubrics.

The Number of Steps Match is given by

$$\text{Num Steps Match} = \frac{1}{N} \sum_{i=1}^N S_i \times 100\% \quad (4-62)$$

Weight Sum Accuracy:

Every generated rubric must have step weights that add up to exactly 1.0 because these weights directly determine how marks are distributed. If they do not sum correctly, the grading arithmetic breaks. We allow a small tolerance of $\varepsilon = 0.05$ to account for floating point rounding, but anything outside that range is treated as invalid.

For generated rubric G_i with step weights $\{w_1, w_2, \dots, w_{n_i}\}$, $W_i = \sum_{j=1}^{n_i} w_j$. The validity indicator is given by

$$V_i = \begin{cases} 1 & \text{if } |W_i - 1.0| < \varepsilon \\ 0 & \text{otherwise} \end{cases} \quad (4-63)$$

The Weight Sum Accuracy is given by

$$\text{Weight Sum Accuracy} = \frac{1}{N} \sum_{i=1}^N V_i \times 100\% \quad (4-64)$$

where:

$\varepsilon = 0.05$.

5 IMPLEMENTATION DETAILS

5.1 Mathpix OCR Integration

Optical Character Recognition (OCR) is an important component of our system as it converts handwritten mathematical solutions into a machine-readable (digitized) format. In this project, the Mathpix API is used to recognize both handwritten and printed mathematical expressions and transform them into LaTeX representations. The generated LaTeX output is then utilized for further parsing and symbolic evaluation within the system.

5.1.1 Loading environment variables

For security purposes, the Mathpix API credentials are stored as environment variables rather than being directly written into the source code. These credentials consist of the Application ID and Application Key required for API authentication. The `load_dotenv()` function is used to load variables from the `.env` file into the application environment, and the values are retrieved using the `os.getenv()` method. This method helps to protect sensitive information and follows standard security practices during API integration.

5.1.2 Initialization of the Mathpix client

After loading the environment variables the next step is to initialize Mathpix client. It is initialized using the above obtained application ID and application key. This client acts as the interface between the system and the Mathpix OCR service. The initialization process is wrapped inside a try-except block. It helps to handle potential runtime errors such as missing credentials, invalid keys, or network-related issues. In case there is any exception then an appropriate error message is displayed and the client variable is assigned `None`. This prevents unexpected system failures.

5.1.3 Calling Mathpix API for OCR

Once the Mathpix client is successfully initialized, it is used to perform OCR on the uploaded image containing handwritten or printed mathematical content. The image file path is passed to the `image_to_text` method of the Mathpix client. The API is configured to return output in LaTeX-styled mathematical notation as well as plain text. Additionally, the `idiomatic_eqn_arrays` option is enabled to ensure that complex multi-line equations are returned in a structured and readable LaTeX format. The resulting LaTeX output serves as the primary input for subsequent modules, including LaTeX pre-processing, step extraction, and symbolic parsing.

5.2 Preprocessing Pipeline

This module processes a LaTeX-formatted mathematical solution and extracts individual solution steps. It removes LaTeX-specific syntax, normalizes mathematical expressions, and outputs a clean, step-wise representation that can be used for evaluation and grading. Ultimately, we obtain an array of handwritten steps in LaTeX representation.

5.2.1 Algorithm

Algorithm 1 Extracting and Cleaning LaTeX Solution Steps

- 1: **Start**
 - 2: Read the LaTeX solution string (The algorithm takes a LaTeX string that represents a mathematical solution written in aligned format)
 - 3: Identify individual solution steps (Each mathematical step in LaTeX is typically written after the alignment symbol &)
 - 4: Scan the string and extracts all parts that appear after the alignment symbol
 - 5: Remove LaTeX formatting commands as extracted steps may contain LaTeX commands such as text formatting, line breaks, or environment declarations
 - 6: Normalize mathematical notation i.e. Convert implicit multiplication (e.g., $2x$) into explicit form ($2 * x$) for further processing
 - 7: Remove any empty/invalid entries created during the cleaning process
 - 8: Return cleaned steps
 - 9: **End**
-

5.2.2 Flowchart

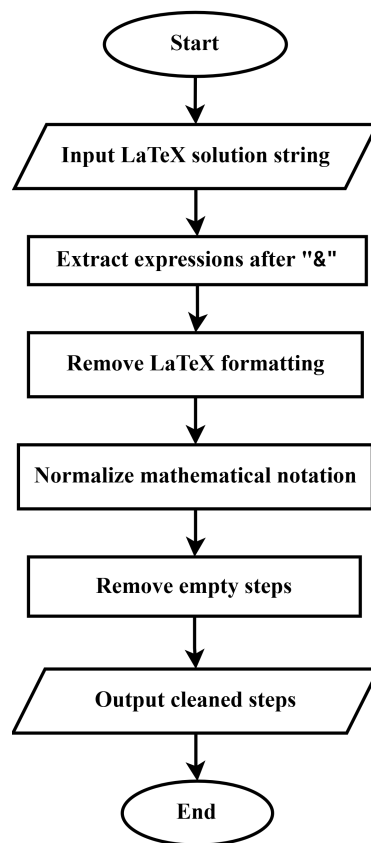


Figure 5-1: Preprocessing Flowchart

5.3 Mathematical Parser

This algorithm converts a cleaned mathematical step (obtained after LaTeX preprocessing) into a symbolic representation using SymPy. Depending on whether the step represents an equation or a single expression, it returns an appropriate symbolic object (SymPy object) that can be used for evaluation and comparison.

5.3.1 Algorithm

Algorithm 2 Parsing a Single Cleaned Mathematical Step

```
1: Start
2: Receive a parsed mathematical step
3: The algorithm takes a single step from the cleaned array of handwritten steps
4: Check for the presence of an equality sign '='
5: if the step contains '=' then
6:     Split the equation into LHS and RHS
7:     Remove leading and trailing whitespace from both sides
8:     if both LHS and RHS are present then
9:         Convert each side into a SymPy expression and simplify
10:    else if only one side is present then
11:        Convert and simplify the available side
12:    else
13:        Mark the step as invalid
14:    end if
15: else
16:     Treat the step as a standalone mathematical expression
17: end if
18: Create a symbolic representation
19: if the step is a valid equation then
20:     Construct a SymPy Eq object
21: else
22:     Return the simplified SymPy expression
23: end if
24: Return the parsed result (symbolic form of the step)
25: End
```

5.3.2 Flowchart

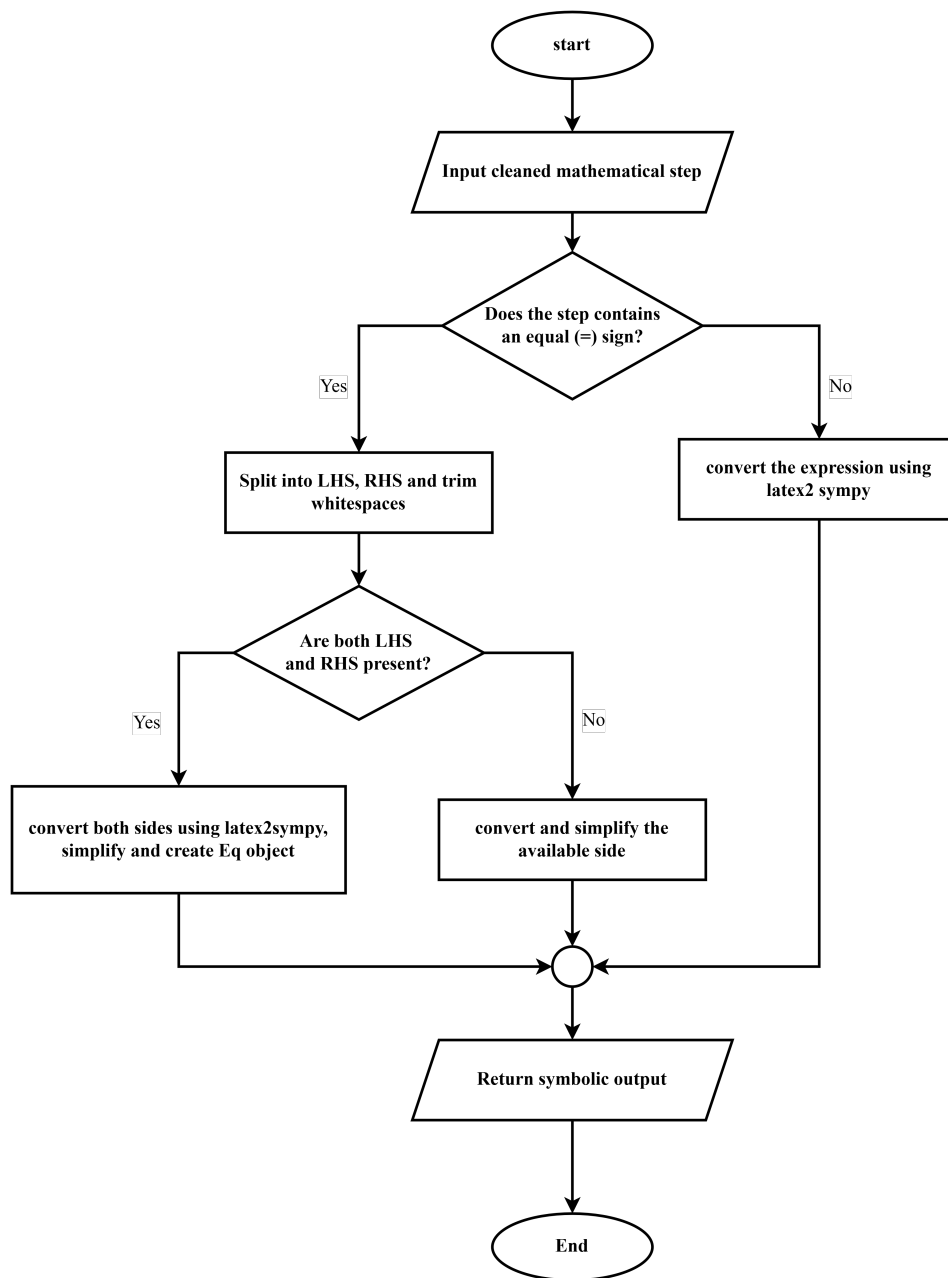


Figure 5-2: Mathematical Parser Flowchart

5.4 Symbolic Reasoning Engine

This module evaluates a sequence of mathematical solution steps obtained from OCR preprocessing. Each step is parsed into symbolic form and compared with the previous step using symbolic reasoning. The system generates a structured report indicating the validity of each step along with explanatory comments. Once an incorrect step is detected, subsequent steps are marked invalid to maintain logical consistency in grading.

5.4.1 Algorithm

This Algorithm 3, evaluates a sequence of mathematical solution steps obtained after OCR and preprocessing. Each step is parsed, validated against the previous step using symbolic reasoning, and recorded in a structured report indicating whether the step is correct or incorrect.

Algorithm 3 Validation of Stepwise Mathematical Solutions

```
1: Start
2: Initialize result storage
3: Create an empty list to store the validation report
4: Parse the first solution step
5: Convert the first step into symbolic form
6: Mark it as valid (represents the initial equation)
7: Add it to the result list with the comment "Initial equation"
8: Initialize error flag
9: Set an error_found flag to track whether an incorrect step has occurred
10: for each remaining step starting from the second one do
11:     Check for previous error
12:     if error_found is true then
13:         Mark the current step as invalid
14:         Add a comment: "Validation cannot continue due to previous error"
15:         Skip further processing of this step
16:         continue
17:     end if
18:     Parse the current step
19:     if parsing fails then
20:         Mark the step as invalid
21:         Add comment: "Parsing failed"
22:         Set the error flag
23:         continue
24:     end if
25:     Validate logical correctness
26:     Compare the current step with the previous valid step using symbolic equivalence
27:     Store validation result
28:     if transformation is valid then
29:         Mark the step as valid
30:         Add comment: "Valid step"
31:         Update reference to previous step
32:     else
33:         Mark the step as incorrect
34:         Add comment: "Incorrect transformation"
35:         Set the error flag
36:     end if
37: end for
38: Return the validation report
39: Output the complete list containing step-wise validation results
40: End
```

5.4.2 Flowchart

The Flowchart 5-3 is for the working of the Symbolic Reasoning Engine.

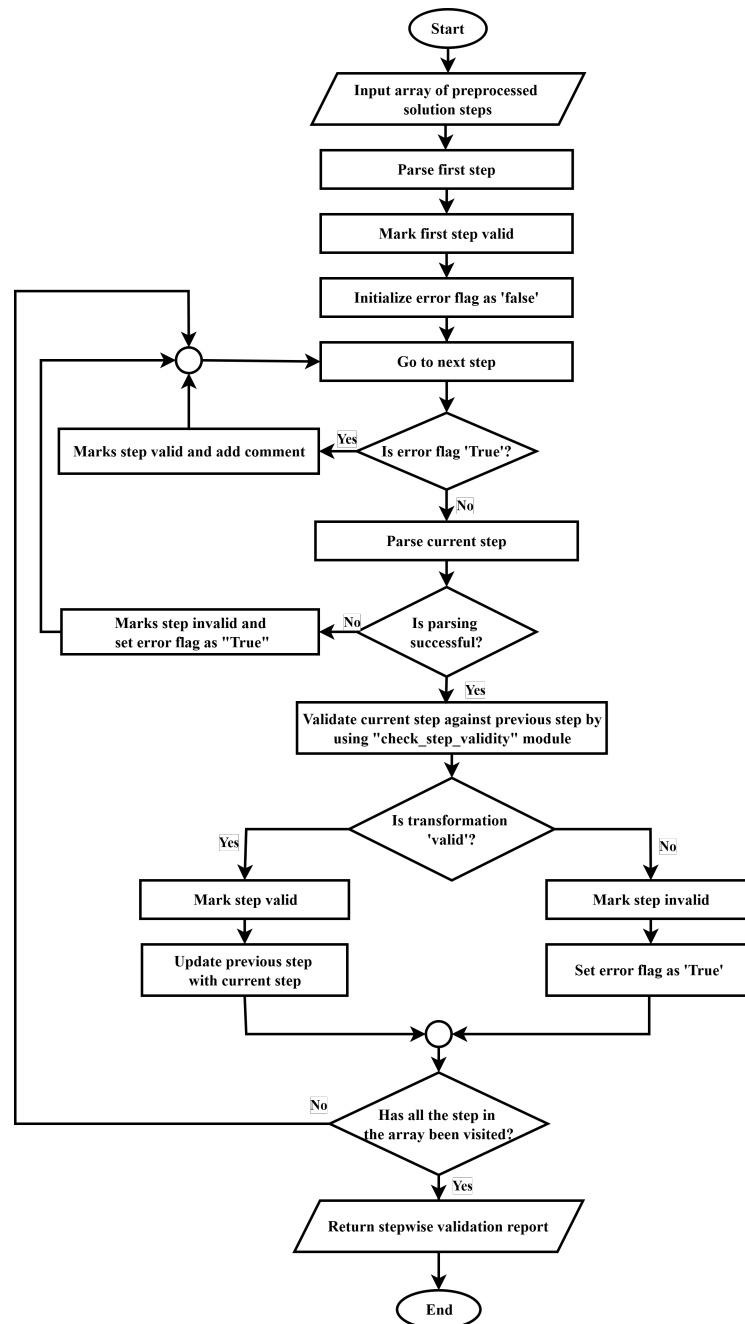


Figure 5-3: symbolic Reasoning Engine Flowchart

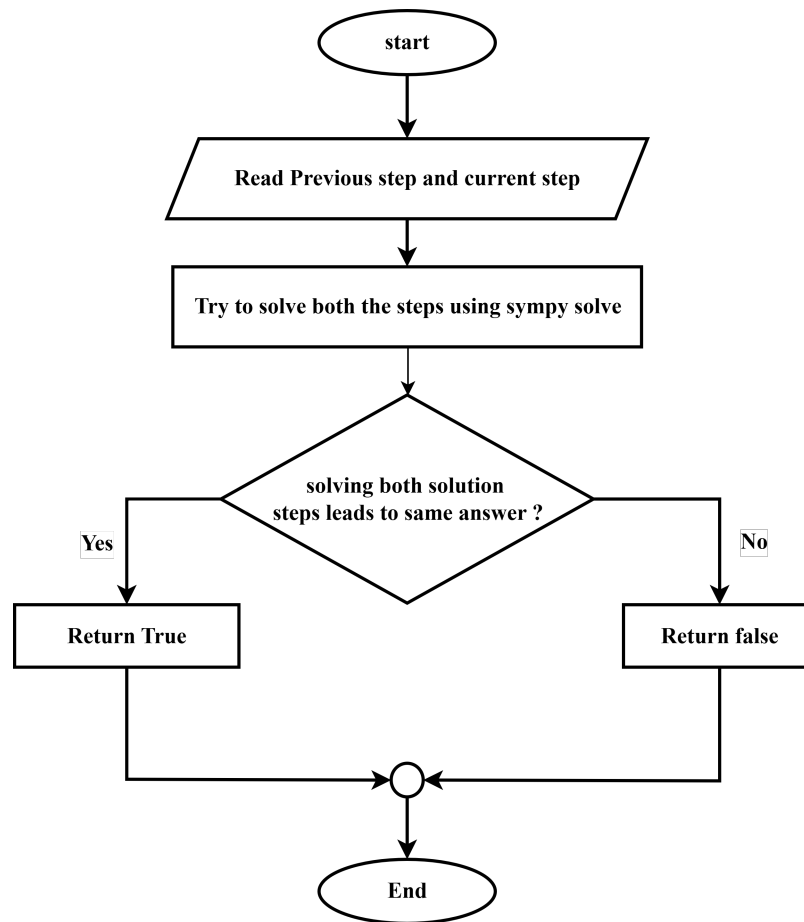


Figure 5-4: Step Validation Module Flowchart

A `check_step_validity` module is used inside the symbolic reasoning engine, which checks if the current equation logically follows from the previous equation. This module returns true if the step sets are the same. The working of this module is represented in the flowchart 5-4.

5.5 DeepSeekMath Model Training

The DeepSeekMath 7B model was fine-tuned using Google Colab with access to an NVIDIA A100 GPU. Fine-tuning was performed using parameter-efficient adaptation to specialize the pretrained model for mathematical question evaluation and rubric-based grading.

5.5.1 Dataset Preparation

The training dataset consisted of 2,797 samples stored in JSON format, where each sample comprised a pair of a mathematical question and its corresponding marking rubric. Prior to training, the dataset was preprocessed and split into training, validation, and testing subsets using an 80:10:10 ratio.

The resulting distribution of samples was as follows:

- Training set: 2,237 samples (80%)
- Validation set: 280 samples (10%)
- Test set: 280 samples (10%)

This data split allowed most of the samples to be used for training. The separate validation and test sets were kept aside for hyperparameter tuning and unbiased performance evaluation.

5.5.2 Training Configuration

Low-Rank Adaptation (LoRA) was used to fine-tune the DeepSeekMath 7B model efficiently. LoRA significantly reduce memory and computational requirements. A LoRA rank value of 32 was selected for training, resulting in 74,956,800 trainable parameters out of the model's total 6,985,322,496 parameters.

The training process was carried out for up to five epochs. To minimize overfitting, early stopping was enabled, and training was stopped whenever the validation loss failed to improve for five consecutive epochs. For optimization, the adamw optimizer was used because of its stable convergence behavior and effective handling of weight decay during training.

The main hyperparameters used during training are listed below:

- LoRA rank: 32

- Number of epochs: 5
- Early stopping patience: 5
- Learning rate: 0.0005
- Optimizer: adamw
- Batch size: 32
- Weight decay: 0.01
- Maximum gradient norm: 1.0

During training, gradient clipping was used with a maximum norm of 1.0. It keeps the learning process stable and prevent gradients from becoming excessively large.

5.5.3 Loss Function

The model was trained using the cross-entropy loss function as it is widely used for autoregressive language modeling tasks. Cross-entropy loss measures the divergence between the predicted probability distribution and the ground truth target distribution over the vocabulary.

Let y_t denote the ground truth token at time step t . Also let $p_{\theta}(y_t | y_{<t})$ represent the probability assigned by the model (parameterized by θ) to the correct token given the previous context. The cross-entropy loss is defined as:

$$\mathcal{L}_{CE} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t}) \quad (5-1)$$

where T is the total number of tokens in the target sequence. Minimizing this loss encourages the model to assign higher probabilities to correct tokens, thereby improving its ability to generate accurate rubric-aligned mathematical evaluations.

6 RESULT AND ANALYSIS

6.1 Frontend Development

We completed the frontend part using React. **Figure 6-1** shows the home page of our system. This frontend is capable of reading handwritten or printed solution steps and questions. **Figure 6-2** shows how to upload the image of the handwritten question and solution. Input validation techniques have also been implemented to avoid invalid input format. This frontend client makes api call to Mathpix OCR via Mathpix SDK and renders the obtained LaTeX in a presentable format.

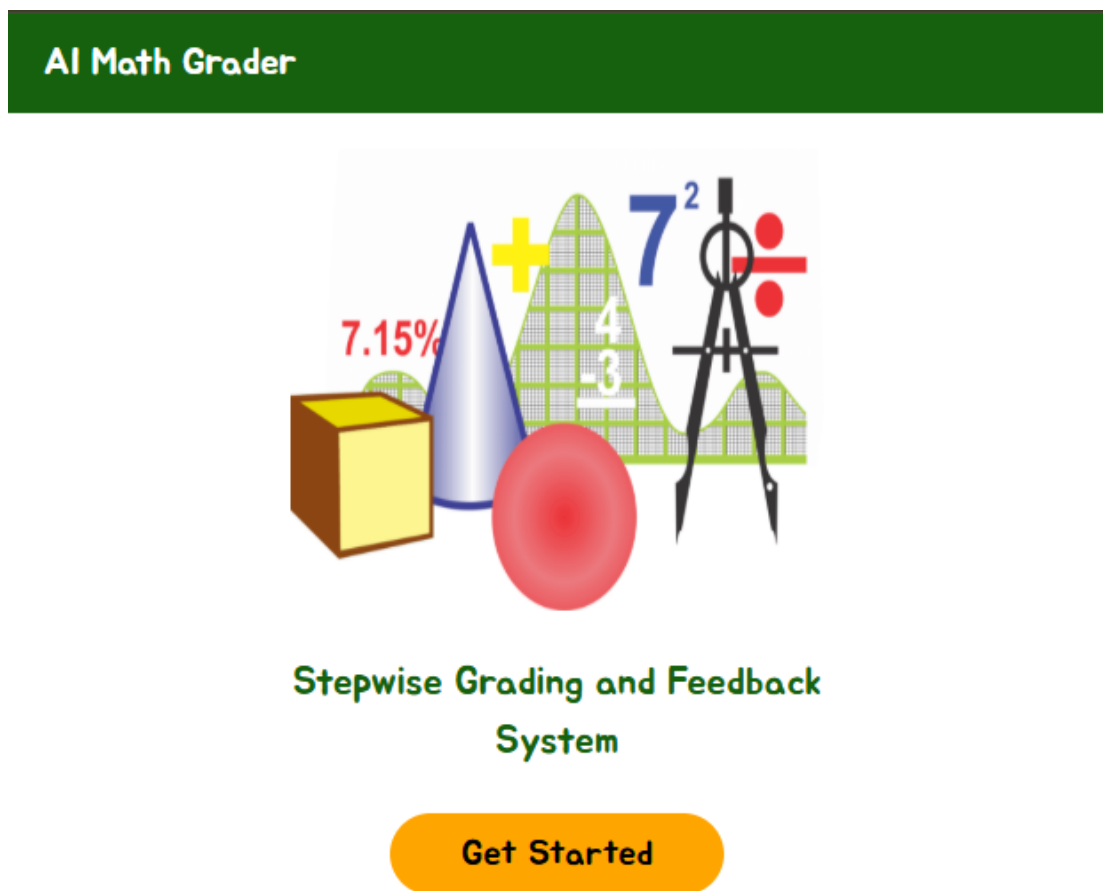
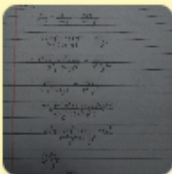


Figure 6-1: Home page

AI Math Grader

Enter Handwritten Question and Answer

Selected: WhatsApp Image 2025-12-04 at 23.27.16_530fbd5c.jpg



Remove

Submit

Figure 6-2: Upload Page

Figure 6-3 shows the $\text{\LaTeX}$ rendering done by the frontend which it received through callback function to Mathpix via Mathpix SDK.

8 simplify $\frac{x}{x-y} - \frac{x}{x+y} + \frac{2xy}{x^2+y^2}$ **Solution,**

$$\begin{aligned} &= \frac{x}{x-y} - \frac{x}{x+y} + \frac{2xy}{x^2+y^2} \\ &= \frac{x(x+y) - x(x-y)}{(x-y)(x+y)} + \frac{2xy}{x^2+y^2} \\ &= \frac{x^2+xy-x^2+xy}{(x-y)(x+y)} + \frac{2xy}{x^2+y^2} \\ &= \frac{2xy}{(x-y)(x+y)} + \frac{2xy}{x^2+y^2} \\ &= \frac{2xy(x^2+y^2) + 2xy(x^2-y^2)}{(x^2-y^2)(x^2+y^2)} \\ &= \frac{2x^3y + 2xy^3 + 2x^3y - 2xy^3}{(x^2-y^2)(x^2+y^2)} \\ &= \frac{4x^3y}{x^4-y} \end{aligned}$$

Ans,

Start Grading

Figure 6-4 shows preprocessing results, Figure 6-5 shows mathematical parser outcomes, Figure 6-6 shows symbolic reasoning engine outputs from the backend, which can be rendered into the Frontend.

[illegible]

Figure 6-4: Preprocessed output

```

-x/(x + y) + x/(x - y) + (2*x*y)/(x**2 + y**2)
(2*x*y)/(x**2 + y**2) + (-x*(x - y) + x*(x + y))/(((x - y)*(x + y)))
(2*x*y)/(x**2 + y**2) + (-x**2 + x**2 + x*y + x*y)/(x**2 - y**2)
(2*x*y)/(x**2 + y**2) + (2*x*y)/(x**2 - y**2)
(2*x*y*(x**2 + x**2 - y**3 + y**2))/(((x**2 - y**2)*(x**2 + y**2)))
(2*x*y*2*x**2)/(x**4 - y**4)
(4*x**3*y)/(x**4 - y**4)

```

Figure 6-5: Parser output

```

[{'step': '\\frac{x}{x-y}-\\frac{x}{x+y}+\\frac{2*x}{x^2+y^2}',
  'valid': True, 'comment': 'Initial equation'},
 {'step': '\\frac{x}{x+y}-x(x-y)/(x-y)(x+y)+\\frac{2*x}{x^2+y^2}',
  'valid': True, 'comment': 'Valid step'},
 {'step': '\\frac{x^2+x-y-x^2+x}{x^2-y^2}+\\frac{2*x}{x^2+y^2}',
  'valid': True, 'comment': 'Valid step'},
 {'step': '\\frac{2*x}{x^2-y^2}+\\frac{2*x}{x^2+y^2}',
  'valid': True, 'comment': 'Valid step'},
 {'step': '\\frac{2*x}{x^2-y^2+\\frac{2*x^2}{x^2-y^2}}+\\frac{2*x}{x^2+y^2}',
  'valid': False, 'comment': 'Incorrect transformation'},
 {'step': '\\frac{2*x}{x^2-y^2+\\frac{2*x^2}{x^2-y^2}}+\\frac{2*x}{x^2+y^2}',
  'valid': False, 'comment': 'Previous error: cannot validate this step'},
 {'step': '\\frac{4*x^3*y}{x^4-y^4}',
  'valid': False, 'comment': 'Previous error: cannot validate this step'}]

```

Figure 6-6: Symbolic reasoning engine output

6.1.1 Output from Symbolic Reasoning Engine

Figure 6-7 shows the transformation of a raw image containing the student's handwritten solutions, after passing through the Symbolic reasoning engine, to its symbolic validation for the correctness of every transition. The output shows that the first step of the solution is indeed correct; however, the second step is incorrect, as the transformation from step 1 to step 2 is invalid (not symbolically equivalent). Also, the error gets propagated to step 3 and step 4, making the final answer incorrect.

Mathematical Validation

Detailed breakdown of step-by-step logic.

$\frac{8^x}{8^{0.5}} = 8^{-2}$	
$8^{x-1/2} = 8^2$	ERROR Invalid transformation - equations are not equivalent
$x - 1/2 = 2$	ERROR Previous error: cannot validate this step
$x = 2.5$	ERROR Previous error: cannot validate this step

Figure 6-7: Symbolic reasoning engine's result for a sample image with an algebraic expression with an invalid step and error propagation

In this Figure 6-7, the symbolic transformation between the first step and the second step is not valid. In the RHS side, there must be 8^{-2} instead of 8^2 . This transform error gets propagated after the second step to the end step.

Similarly, **Figure 6-8, 6-9, 6-10** shows the corresponding results with varying nature of inputs.

Mathematical Validation

Detailed breakdown of step-by-step logic.

$$\frac{8^x}{\sqrt{8}} = \frac{1}{64}$$

$$\frac{8^x}{8^{0.5}} = 8^{-2}$$

$$8^{x-0.5} = 8^{-2}$$

$$x - 0.5 = -2$$

$$x = -2 + 0.5$$

$$x = -1.5$$

$$x = -3/2$$

Figure 6-8: Symbolic reasoning engine's result for a sample image with an algebraic expression with all correct solution steps

In this Figure 6-8, the symbolic transformation is correct between all steps. As a result, all the boxes containing the steps are green.

Mathematical Validation

Detailed breakdown of step-by-step logic.

$$\log(x-1) + \log(x+1) = \log_2 1$$

$$\log(x-1) + \log(x+1) = 0$$

$$\log[(x-1)(x+1)] = 0$$

$$\log[(x-1)(x+1)] = \log 1$$

$$(x-1)(x+1) = 1$$

$$x^2 - 1 = 1$$

$$x^2 = 2$$

$$x = \pm 2$$

ERROR
Invalid transformation - equations are not equivalent
(checking 2 branches)

Figure 6-9: Symbolic reasoning engine's result for a sample image with a logarithmic expression with an incorrect final answer

Our symbolic reasoning engine caught an error in the transformations of the last step in Figure 6-9. It should have been $x = \pm\sqrt{2}$ instead of $x = \pm 2$ in the last step.

Mathematical Validation

Detailed breakdown of step-by-step logic.

$$\log(x-1) + \log(x+1) = \log_2 1$$

$$\log(x-1) + \log(x+1) = 0$$

$$\log[(x-1)(x+1)] = 0$$

$$\log[(x-1)(x+1)] = \log 1$$

$$(x-1)(x+1) = 1$$

$$x^2 - 1 = 1$$

$$x^2 = 2$$

$$x = \pm\sqrt{2}$$

Figure 6-10: Symbolic reasoning engine's result for a sample image with a logarithmic expression with all correct intermediate steps as well as the final answer

Mathematical Validation

Detailed breakdown of step-by-step logic.

$$\frac{1}{2(x-y)} - \frac{1}{2(x+y)} - \frac{y}{(x+y)(x-y)}$$

$$\frac{(x+y) - (x-y)}{2(x+y)(x-y)} - \frac{y}{(x+y)(x-y)}$$

$$\frac{x+y-x+y-3*y}{2(x+y)(x-y)}$$

ERROR
Invalid transformation - equations are not equivalent

$$\frac{2*y-2*y}{2(x+y)(x-y)}$$

ERROR
Previous error: cannot validate this step

$$\frac{0}{2(x+y)(x-y)}$$

ERROR
Previous error: cannot validate this step

$$0$$

ERROR
Previous error: cannot validate this step

Figure 6-11: Symbolic reasoning engine's result for a sample image with an algebraic expression with some incorrect steps

In the third step, the student wrote $3xy$ instead of $2xy$, i.e., numerical value error. Due to which the error propagated from the 3rd step to the last step as shown by the red boxes in Figure 6-11.

Mathematical Validation

Detailed breakdown of step-by-step logic.

$$\frac{1}{2(x-y)} - \frac{1}{2(x+y)} - \frac{y}{(x+y)(x-y)}$$

$$\frac{(x+y) - (x-y)}{2(x+y)(x-y)} - \frac{y}{(x+y)(x-y)}$$

$$\frac{x+y-x+y-2*y}{2(x+y)(x-y)}$$

$$\frac{2*y-2*y}{2(x+y)(x-y)}$$

$$\frac{0}{2(x+y)(x-y)}$$

$$0$$

Figure 6-12: Symbolic reasoning engine's result for a sample image with an algebraic expression with correct steps

6.2 Train/Validation loss curve

Figure 6-13 shows the loss curves after model training. We can see both the training and validation loss decrease with respect to epochs. The training loss drops sharply in the initial steps, indicating rapid learning as the model fits the data. The validation loss follows a very similar trend and closely tracks the training loss, which suggests good generalization. After the early phase, both curves flatten gradually, showing the model is approaching convergence and learning more slowly. The small, stable gap between training and validation loss indicates no significant over-fitting. Final low and steady losses imply the model has converged to a stable optimum.

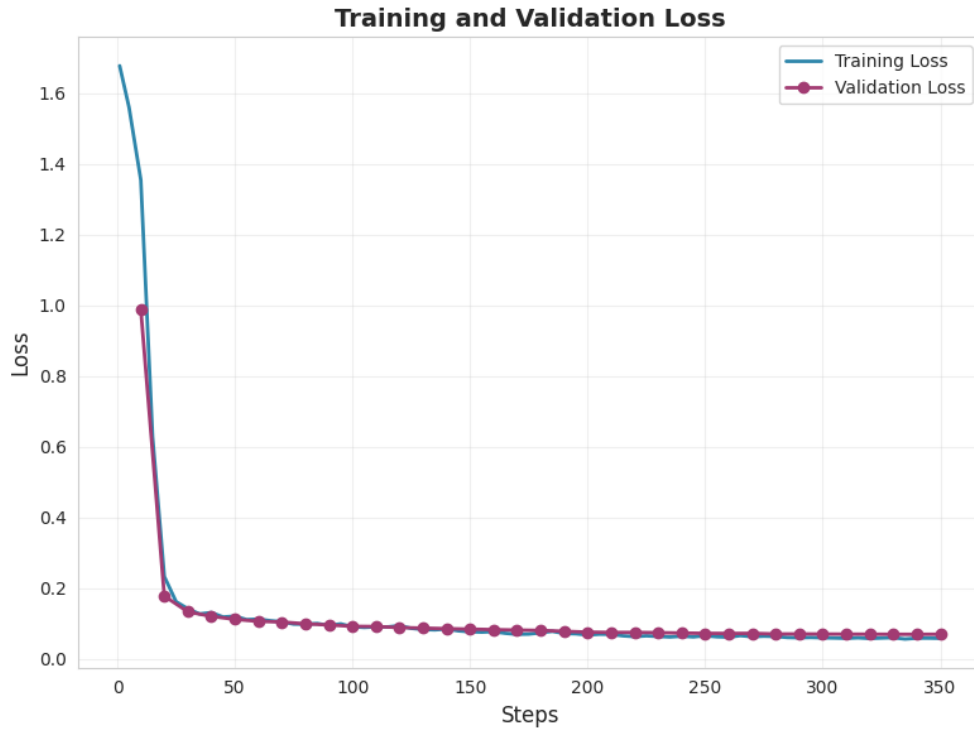


Figure 6-13: Loss Curve

6.3 Model Results

Test was performed on a small set of 3 new questions due to the long time taken by our fine-tuned model. Here are the obtained results.

Input Questions

The following questions were provided as input to the model:

Table 6-1: New questions provided to the fine-tuned DeepSeekMath model.

ID	Question Text	Topic	Difficulty	Max Marks
Q1	Solve: $\log_a n = \log_a 3 + \log_a (2n - 1)$	LOG	1	3
Q2	$\frac{3x+1}{9x^2+3x+1} + \frac{3x-1}{9x^2-3x+1} + \frac{2}{81x^4+9x^2+1}$	SIM	1	3

Generated Rubrics

The fine-tuned model generated the following rubrics for the input questions. Each rubric specifies the expected answer forms, step identifiers, weights, and whether the step is final.

Table 6-2: Generated rubrics for the new questions after fine-tuning.

Q-ID	Expected Forms	Weight	Step ID	Final Step
Q1	$n = 6n - 3$	0.333	S1	No
	$= \frac{3}{5}$	0.667	S2	Yes
Q2	$= \frac{(3x+1)(9x^2-3x+1) + (3x-1)(9x^2+3x+1)}{(9x^2+3x+1)(9x^2-3x+1)}$	0.333	S1	No
	$+ \frac{2}{81x^4+9x^2+1}$			
	$= \frac{54x^3+2}{81x^4+9x^2+1}$	0.333	S2	No
	$= \frac{2(3x+1)}{9x^2+3x+1}$	0.334	S3	Yes

Observations

From the results in Table 6-2, it is observed that:

- The model successfully adapted to the rubric format.
- However, the generated rubrics did not exactly match the rubrics in the test set.
- This indicates limited generalization of the fine-tuned model.
- Further tuning of hyperparameters and training with more diverse examples is necessary to improve rubric accuracy.

Overall, while the model demonstrates the ability to produce structured rubrics, there remains scope for optimization to achieve reliable auto-grading performance.

6.4 Output obtained from Marking and Feedback Engine

The results obtained from the marking and feedback engine for various input images are shown in this section.

Figure 6-14 shows all correct solution steps is marked full (3/3) whereas Figure 6-15 received partial marks (1/3) as there is incorrect intermediate steps. Similarly, Figure 6-16 and Figure 6-17 also award partial markings for different question types.

Mathematical Validation
Detailed breakdown of step-by-step logic and teacher feedback.

FINAL SCORE
3 / 3

PROBLEM STATEMENT

$$Q\text{Solve } \frac{8^x}{\sqrt{8}} = \frac{1}{64}$$

MAX MARKS DIFFICULTY

3

Hard

$$\frac{8^x}{\sqrt{8}} = \frac{1}{64}$$

FEEDBACK
It's good practice to start by clearly writing down the problem.
This is a correct restatement of the given equation.

$$\frac{8^x}{8^{0.5}} = 8^{-2}$$

FEEDBACK
Excellent work recognizing the common base and converting the terms. This is a key step in solving exponential equations!
You have correctly converted both sides of the equation to a common base of 8 by expressing $\sqrt{8}$ as $8^{0.5}$ and $\frac{1}{64}$ as 8^{-2} .

+1 Marks

$$8^{x-0.5} = 8^{-2}$$

FEEDBACK
Well done! Applying exponent rules correctly simplifies the equation significantly.
You have correctly applied the exponent rule for division with the same base, combining 8^x and $8^{0.5}$ into $8^{x-0.5}$.

+1 Marks

$$x - 0.5 = -2$$

FEEDBACK
Great job moving from exponential form to a linear equation. You're almost there!
You have correctly equated the exponents after establishing a common base, which is the next logical step to solve for x.

+0.5 Marks

$$x = -2 + 0.5$$

FEEDBACK
Good algebra! Keep up the precision.
You have correctly isolated the variable x by adding 0.5 to both sides of the equation.

$$x = -1.5$$

FEEDBACK
Fantastic! You've successfully found the value of x.
You have correctly calculated the value of x.

+0.5 Marks

$$x = -3/2$$

FEEDBACK
It's great to see you can present your answer in different equivalent forms, both decimal and fractional.
You have correctly expressed the answer in fractional form.

FINAL TEACHER'S SUMMARY

"Excellent work! You have successfully solved the equation by correctly applying exponent rules and algebraic manipulation. All steps are accurate and well-executed."

Figure 6-14: A sample image with all solution steps correct receives full marks along with feedback.

Mathematical Validation

detailed breakdown of step-by-step logic and teacher feedback.

FINAL SCORE

1 / 3

PROBLEM STATEMENT

$$Q\text{Solve } \frac{8^x}{\sqrt{8}} = \frac{1}{64}$$

MAX MARKS DIFFICULTY

3

Hard

$$\frac{8^x}{8^{0.5}} = 8^{-2}$$

FEEDBACK

Excellent start! You've successfully applied the rules of exponents and roots to rewrite the equation in a much clearer form. This is a crucial first step in solving such problems.

The transformation of the original equation into an equivalent form with a common base of 8 on both sides is correct. You correctly converted $\sqrt{8}$ to $8^{0.5}$ and $\frac{1}{64}$ to 8^{-2} .

+1 Marks

$$8^{x-1/2} = 8^2$$

ERROR

You're very close! Remember to be careful with negative signs when carrying values from one step to the next. Double-check your work, especially when dealing with inverse operations or exponents. Keep up the great effort!

While you correctly combined the exponents on the left side (dividing powers with the same base means subtracting their exponents), there is an error in the exponent on the right side. In the previous step, the right side was 8^{-2} , but here it incorrectly became 8^2 . The negative sign was omitted.

$$x - 1/2 = 2$$

ERROR

You demonstrated good understanding of equating exponents when bases are the same. That's a strong mathematical principle! Just make sure to build on correct previous steps. A small error can sometimes lead to a ripple effect. Reviewing your work for small errors, like missing negative signs, can really help.

This step correctly equates the exponents from the previous (incorrect) step. However, because the right-hand side exponent in step 2 was incorrect (it should have been -2, not 2), this equation is also incorrect relative to the original problem.

$$x = 2.5$$

ERROR

Your algebraic manipulation to isolate x is precise given the equation you were working with. That's a valuable skill! Continue to focus on accuracy in every step, especially with signs, as even minor errors can change the final solution. You're showing strong potential!

The arithmetic to solve for x from the equation $x - \frac{1}{2} = 2$ is correct. However, since the equation it derived from (Step 3) was incorrect due to the error in Step 2, this final answer is not correct for the original problem.

FINAL TEACHER'S SUMMARY

"The student demonstrated a good initial understanding of exponent rules to transform the equation but made a critical sign error in the second step, which propagated through the rest of the solution. This resulted in an incorrect final answer."

Figure 6-15: A sample image with incorrect intermediate solution steps receives partial marks along with feedback.

Mathematical Validation

Detailed breakdown of step-by-step logic and teacher feedback.

FINAL SCORE

0 / 2

PROBLEM STATEMENT

$$RSolve \log_6 x = 2$$

MAX MARKS DIFFICULTY

2

Medium

$$\log_6 x = 2$$

FEEDBACK

You've accurately written down the problem! Now, let's think about how to transform this logarithmic equation into an exponential one to solve for x .

This step correctly restates the given equation.

$$x = 2^6$$

ERROR

Remember the definition of a logarithm: $\log_b x = y$ is equivalent to $x = b^y$. Double-check which number is the base (b) and which is the result (y) in your transformation.

The transformation from logarithmic to exponential form is incorrect. The base of the logarithm should be the base of the exponent in the exponential form.

$$x = 64$$

ERROR

Your calculation of 2^6 is accurate! It's great to see your arithmetic skills. Let's focus on getting the setup right in the previous step, and then your strong calculation abilities will lead you to the correct final answer.

The calculation $2^6 = 64$ is numerically correct. However, this step is based on an incorrect transformation in the previous step.

FINAL TEACHER'S SUMMARY

"The student did not correctly solve the problem. The main error was in the transformation from the logarithmic form to the exponential form, which led to an incorrect final answer."

Figure 6-16: A sample image with incorrect intermediate logarithmic solution steps receives partial marks along with feedback.

Mathematical Validation
 Detailed breakdown of step-by-step logic and teacher feedback.

FINAL SCORE
3.25 / 4

PROBLEM STATEMENT

Q Solve $\log(x-1) + \log(x+1) = \log_2 1$

MAX MARKS 4
 DIFFICULTY Not specified

$$\log(x-1) + \log(x+1) = \log_2 1$$

FEEDBACK

Excellent start! You've correctly transcribed the problem.
The initial equation is correctly written down from the problem statement.

+0.25 Marks

$$\log(x-1) + \log(x+1) = 0$$

FEEDBACK

Well done! Simplifying $\log_2 1$ to 0 is a great first step in solving this equation.
The simplification of $\log_2 1$ to 0 is correct, as any logarithm of 1 is 0.

+0.5 Marks

$$\log[(x-1)(x+1)] = 0$$

FEEDBACK

Great use of the logarithm product rule! This simplifies the equation nicely.
The logarithm property $\log A + \log B = \log(AB)$ is correctly applied to combine the terms.

+0.5 Marks

$$\log[(x-1)(x+1)] = \log 1$$

FEEDBACK

Smart move to express 0 as $\log 1$. This sets you up perfectly for the next step.
The number 0 is correctly expressed as $\log 1$ (assuming base 10, which is implied by the lack of a subscript).

+0.5 Marks

$$(x-1)(x+1) = 1$$

FEEDBACK

Excellent job moving from the logarithmic form to an algebraic equation!
By equating the arguments of the logarithms (since $\log A = \log B$ implies $A = B$), this step correctly eliminates the logarithm.

+0.5 Marks

$$x^2 - 1 = 1$$

FEEDBACK

You've correctly expanded the binomial product. Keep up the good work!
The product $(x-1)(x+1)$ is correctly expanded to $x^2 - 1$ using the difference of squares formula.

+0.5 Marks

$$x^2 = 2$$

FEEDBACK

Good algebraic manipulation! You're very close to finding the solution for x .
The constant term is correctly moved to the right side of the equation, isolating x^2 .

+0.5 Marks

$$x = \pm 2$$

ERROR

Be careful when taking the square root. Remember that if $x^2 = 2$, then $x = \pm\sqrt{2}$, not ± 2 . Also, don't forget to check your solutions against the domain of the original logarithmic functions!
When solving $x^2 = 2$, the correct solution is $x = \pm\sqrt{2}$, not $x = \pm 2$. This is an arithmetic error in taking the square root.

Figure 6-17: A sample image with incorrect intermediate solution steps receives partial marks along with feedback.

6.5 Analysis of Results

- Mathpix OCR can convert most of the mathematical expressions into LaTeX equivalent.
- Preprocessing module works for all multi-step algebraic LaTeX.
- Mathematical parser (latex2sympy) can parse (useful for our case):
 - Algebraic Simplification
 - Algebraic Equation
 - Indices and Logarithmic
 - Certain inconsistencies were observed in parsing either–or type answers because of the Limitations of the Latex2sympy parser.
- The results highly depend on the Sympy latex2sympy library scope.
- Handles stepwise validation (Step transformation error)
- Symbolic reasoning engine works perfectly as long as Sympy works.
- Deepseek Math model adapts in generating rubrics for the questions in JSON format.
- It generates correct rubrics for the question where the intermediate steps are correct.
- For the question which are out of the scope of our dataset, this rubric generation model generates a single-step rubric, which is the final answer.
- the scoring and feedback engine provides correct, logical, and personalized feedback for every question step.
- These scores and feedback are based on the symbolic reasoning engine and rubrics generation model's output. This prevents the LLM from hallucinating and helps in maintaining consistency.

7 CONCLUSION

With the completion of the project, we were able to develop an Automated Marking and Feedback System for multi-step algebraic mathematical solutions by integrating Optical Character Recognition (OCR), symbolic reasoning, and a large language model. The Mathpix OCR module proved effective in converting handwritten and printed mathematical expressions into structured LaTeX format. It can be considered a good decision to use it, as the accuracy of all downstream modules, including the preprocessing pipeline, mathematical parser, and symbolic reasoning engine, is directly dependent on the quality of OCR output. The earlier experimented TrOCR model was found insufficient for complex, poorly written expressions, making the Mathpix API the appropriate choice.

The RegEx-based preprocessing module reliably extracted individual solution steps from raw LaTeX output by performing pattern matching, noise normalization, and step segmentation. This approach acted as a bridge between OCR-based perception and symbolic reasoning. The mathematical parser, using the `latex2sympy2` library, successfully converted LaTeX steps into SymPy symbolic objects for algebraic simplification, algebraic equations, and logarithmic and index expressions.

The Symbolic Reasoning Engine, built on the SymPy Computer Algebra System (CAS), performed stepwise validation by checking symbolic equivalence between consecutive solution steps using tree-based expression representation, canonicalization, rule-based simplification, and exact equation solving. The engine was accurate in detecting invalid algebraic transformations, sign errors, and arithmetic errors, and correctly propagated error flags to subsequent steps once an incorrect transition was identified. The system’s frontend, developed using React, rendered the validation results visually, clearly distinguishing between valid and invalid steps.

The DeepSeekMath-7B model was fine-tuned on a dataset of 2,797 question–rubric samples split in an 80:10:10 ratio for training, validation, and testing. The LoRA technique, with rank 32, helped reduce the memory and computational requirements by keeping most model weights frozen and only training a small set of parameters. The model was trained for up to 5 epochs using the AdamW optimizer with a learning rate of 0.0005 and cross-entropy loss. The training and validation loss curves both decreased steadily and converged closely together, indicating that the model learned well without overfitting.

The system successfully combines symbolic reasoning with a large language model

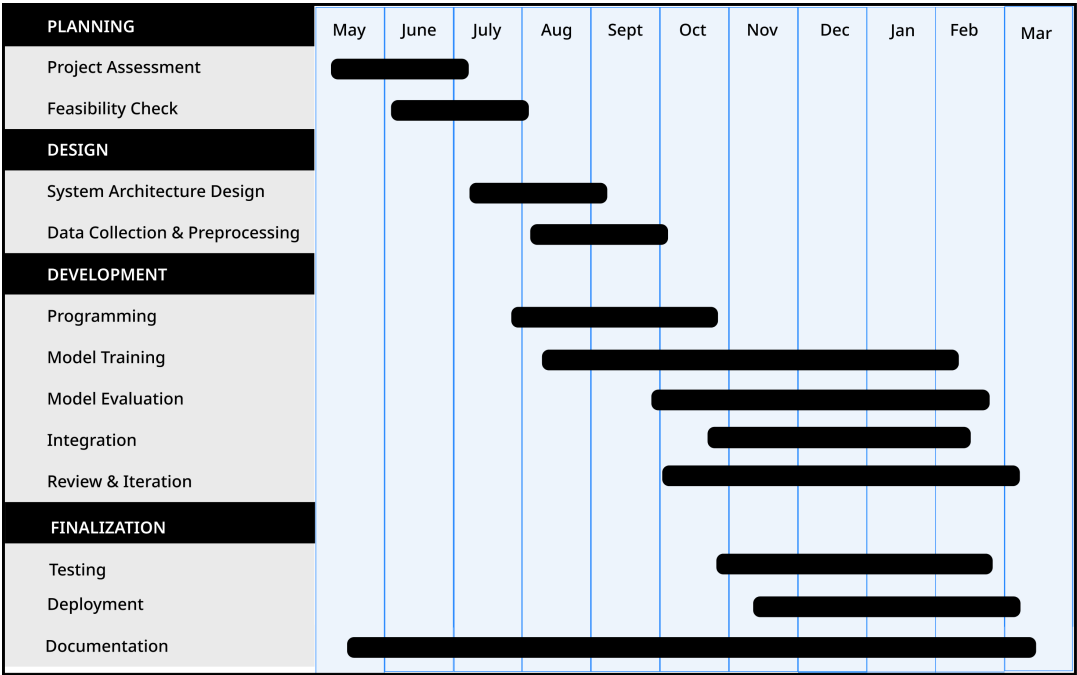
to evaluate multi-step algebra problems, award partial marks, detect step-level errors, and provide feedback. It covers three topic areas: algebraic simplification, algebraic equations, and logarithmic expressions. But, the system currently handles only a limited set of question types, requires reasonably clear handwriting, and may occasionally miss conceptual mistakes that a human examiner would catch. In the future, the system can be improved by adding more question categories, making the OCR more robust to different handwriting styles, and training on a larger and more diverse dataset to produce more accurate and generalizable rubrics.

8 APPENDICES

Appendix A : Gantt Chart

Our primary focus in Part B will be on dataset preprocessing, model fine-tuning, system integration, testing and deployment i.e. finalizing the project. We have a timeline of 3 months starting from December to March.

Table 8-1: Gantt Chart



Appendix B : Project Budget

Table 8-2: Project Budget

SN	Particular	Cost(NPR)
1	Computing Resources (Google Colab premium)	5000
2	Mathpix API	3000
3	Printing and Report binding	2500
4	Miscellaneous (Stationery, Internet/Data Utilities)	1500
	Total	12000

Appendix C : code snippets

```
load_dotenv()
# Loading environment variables
Mathpix_App_ID = os.getenv('Mathpix_App_ID')
Mathpix_App_Key = os.getenv('Mathpix_App_Key')
```

Listing 8: Loading Environment variables

```
# Initialize Mathpix Client
try:
    mathpix_client = MathpixClient(
        app_id=Mathpix_App_ID,
        app_key=Mathpix_App_Key)
except Exception as e:
    print("Error initializing MathpixClient:", e)
    mathpix_client = None
```

Listing 9: Initializing Mathpix Client

```
# Call the Mathpix API to convert the image to LaTeX
api_result = mathpix_client.image_new(
    file_path=filepath,
    formats=['latex_styled', 'text'],
    idiomatic_eqn_arrays=True
)
```

Listing 10: Calling Mathpix API for OCR

```
def preprocessing(latex_str):
    # capturing everything after &
    pattern = r"&(.*)"
    clean_pattern = r'\\text\s*\{.*?\}|\\therefore|\\\\\\\\\\\\\\begin\s*\{.*?\}.*'
    lines = re.findall(pattern, latex_str)
    cleaned_arr = [re.sub(clean_pattern, '', item).strip()
                    for item in lines]
    cleaned_arr = [re.sub(r'(\d)\s*([a-zA-Z])', r'\1*\2',
                          item).strip() for item in cleaned_arr]
    cleaned_arr = [item for item in cleaned_arr if item]
    return cleaned_arr
```

Listing 11: LaTeX Preprocessing Module

```

def parse_math_step(step_str):
    """
    Parses a math step string.
    If it contains '=', returns a SymPy Eq object (if both
    sides exist).
    If not, returns a SymPy expression.
    """
    if '=' in step_str:
        lhs, rhs = step_str.split('=')
        lhs = lhs.strip()
        rhs = rhs.strip()
        if lhs and rhs:
            return Eq(latex2sympy(lhs), latex2sympy(rhs))
        elif lhs: # Only LHS present
            return latex2sympy(lhs)
        elif rhs: # Only RHS present
            return latex2sympy(rhs)
        else:
            return None
    else:
        # Just an expression, not an equation
        return latex2sympy(step_str.strip())

```

Listing 12: Mathematical Parser Module

```

from sympy import solve

def check_step_validity(prev_eq, curr_eq):
    """
    Checks if current equation logically follows from
    previous equation.
    Returns True if solution sets are the same.
    """
    x = symbols('x')
    y = symbols('y')

    try:
        prev_sol = simplify(prev_eq) # or solve
        curr_sol = simplify(curr_eq)
        return prev_sol == curr_sol
    except Exception as e:

```

```
print("Error:", e)
return False
```

Listing 13: Check Step Validity Function

```
def check_steps(steps):
    results = []
    prev_eq = parse_math_step(steps[0])
    results.append({
        "step": steps[0],
        "valid": True,
        "comment": "Initial equation"
    })

    error_found = False

    for i in range(1, len(steps)):
        if error_found:
            results.append(
                {
                    "step": steps[i],
                    "valid": False,
                    "comment": "Previous error: cannot validate
                               this step"
                })
            continue

        curr_eq = parse_math_step(steps[i])
        if curr_eq is None:
            results.append({
                "step": steps[i],
                "valid": False,
                "comment": "Parsing failed"
            })
            error_found = True
            continue

        valid = check_step_validity(prev_eq, curr_eq)
        results.append({
            "step": steps[i],
            "valid": valid,
            "comment": "Valid step" if valid else "Incorrect"
```

```
        transformation"

    })

    if not valid:
        error_found = True
    else:
        prev_eq = curr_eq

    return results
```

Listing 14: Symbolic Reasoning Engine

Appendix D: Details of Rubrics Dataset

The complete dataset contains 2797 samples. Three selected dataset samples were presented here.

```
[
  {
    "question": {
      "text": "\\frac{1}{x+y} - \\frac{1}{x-y} + \\frac{2x}{x^2-y^2}",
      "topic": "SIM",
      "difficulty": 0,
      "max_marks": 2
    },
    "rubric": [
      {
        "step_id": "Q14S1",
        "expected_forms": [
          "= \\frac{x-y-(x+y)}{x^2-y^2} + \\frac{2x}{x^2-y^2}"
        ],
        "weight": 0.25
      },
      {
        "step_id": "Q14S2",
        "expected_forms": [
          "= \\frac{-2y+2x}{x^2-y^2}",
          "= \\frac{2(x-y)}{(x+y)(x-y)}"
        ],
        "weight": 0.25
      }
    ],
    "question": {
      "text": "Solve:  $2^{3x-5} a^{x-2} = 2^{x-2} a^{1-x}$ ",
      "topic": "EQ",
      "difficulty": 1,
      "max_marks": 3
    },
    "rubric": [
      {
        "step_id": "Q122S1",
```

```

    "expected_forms": [
      "= \\frac{2^{{3x-5}}}{{2^{{x-2}}}} = \\frac{{a^{{1-x}}}}{{a^{{x-2}}}}"
    ],
    "weight": 0.333
  },
  {
    "step_id": "Q122S2",
    "expected_forms": [
      "= (2a)^{{2x-3}} = 1"
    ],
    "weight": 0.333
  },
  {
    "step_id": "Q122S3",
    "expected_forms": [
      "= x = \\frac{3}{{2}}"
    ],
    "weight": 0.334,
    "is_final": true
  },
  {
    "question": {
      "text": "\\log_2(8x^2)",
      "topic": "LOG",
      "difficulty": 0,
      "max_marks": 2
    },
    "rubric": [
      {
        "step_id": "Q177S1",
        "expected_forms": [
          "=3+2y"
        ],
        "weight": 1.0,
        "is_final": true
      }
    ]
  }
]

```

Appendix E: Response from Marking and Feedback Engine

```
{
  "question_data": "Q. Solve  $\log_2(x-1) + \log_2(x+1) = \log_2 1$ ",
  "difficulty_level": "Hard",
  "step_evaluations": [
    {
      "step_index": 1,
      "step_content": " $\log_2(x-1) + \log_2(x+1) = \log_2 1$ ",
      "awarded_marks": 0,
      "is_correct": true,
      "technical_critique": "This is the correctly stated original equation.",
      "pedagogical_feedback": "You've accurately written down the problem."
    },
    {
      "step_index": 2,
      "step_content": " $\log_2(x-1) + \log_2(x+1) = 0$ ",
      "awarded_marks": 0.5,
      "is_correct": true,
      "technical_critique": "Correctly evaluated  $\log_2 1$  as 0.",
      "pedagogical_feedback": "Excellent start by simplifying the right-hand side of the equation!"
    },
    {
      "step_index": 3,
      "step_content": " $\log_2[(x-1)(x+1)] = 0$ ",
      "awarded_marks": 1,
      "is_correct": true,
      "technical_critique": "Correctly applied the logarithm product rule  $\log A + \log B = \log(AB)$ .",
      "pedagogical_feedback": "Great application of the logarithm property to combine the terms on the left side!"
    }
  ],
}
```

```

{
  "step_index": 4,
  "step_content": "\\log [(x-1)(x+1)]=\\log 1",
  "awarded_marks": 0.5,
  "is_correct": true,
  "technical_critique": "Correctly expressed 0 as \\( \\log 1 \\) (assuming base 10, which is implied by the absence of a subscript).",
  "pedagogical_feedback": "Smart move to rewrite 0 in terms of a logarithm, which helps in the next step!"
},
{
  "step_index": 5,
  "step_content": "(x-1)(x+1)=1",
  "awarded_marks": 0.5,
  "is_correct": true,
  "technical_critique": "Correctly removed the logarithm from both sides by equating the arguments.",
  "pedagogical_feedback": "You've successfully converted the logarithmic equation into an algebraic one. Keep up the good work!"
},
{
  "step_index": 6,
  "step_content": "x^{2}-1=1",
  "awarded_marks": 0.5,
  "is_correct": true,
  "technical_critique": "Correctly expanded the product \\( (x-1)(x+1) \\) using the difference of squares identity.",
  "pedagogical_feedback": "Well done on expanding the product accurately!"
},
{
  "step_index": 7,
  "step_content": "x^{2}=2",
  "awarded_marks": 0.5,
  "is_correct": true,
  "technical_critique": "Correctly isolated \\( x^2 \\) by adding 1 to both sides of the equation.",

```

```

        "pedagogical_feedback": "You're on track to solving
                                for  $(x)$ !"
    },
    {
        "step_index": 8,
        "step_content": "x=  $\pm \sqrt{2}$ ",
        "awarded_marks": 0,
        "is_correct": false,
        "technical_critique": "While you correctly solved the
                                algebraic equation  $(x^2 = 2)$  to get  $(x =$ 
                                 $\pm \sqrt{2})$ , you missed the critical step of
                                verifying these solutions against the domain of
                                the original logarithmic equation. The terms  $(\log(x-1))$  and  $(\log(x+1))$  require  $(x-1$ 
                                 $> 0)$  and  $(x+1 > 0)$ , which means  $(x > 1)$ 
                                 $)$ . Therefore,  $(x = -\sqrt{2})$  is an
                                extraneous solution and should be rejected. Only
                                 $(x = \sqrt{2})$  is a valid solution.",
        "pedagogical_feedback": "You correctly found the
                                algebraic solutions! However, with logarithmic
                                equations, it's really important to always check
                                your answers to make sure they fit the domain of
                                the original problem. Think about what values of
                                 $(x)$  would make the terms inside the
                                logarithms positive. This will help you find the
                                true solution(s)!"
    }
],
"total_marks_awarded": 3.5,
"final_verdict": "The student demonstrated excellent
                    algebraic skills in solving the equation but overlooked
                    checking for extraneous solutions, a crucial step for
                    logarithmic equations.",
"total_marks": 4
}

```

Listing 15: Response from Marking and Feedback Engine

Appendix F: Plagiarism Report Summary Page

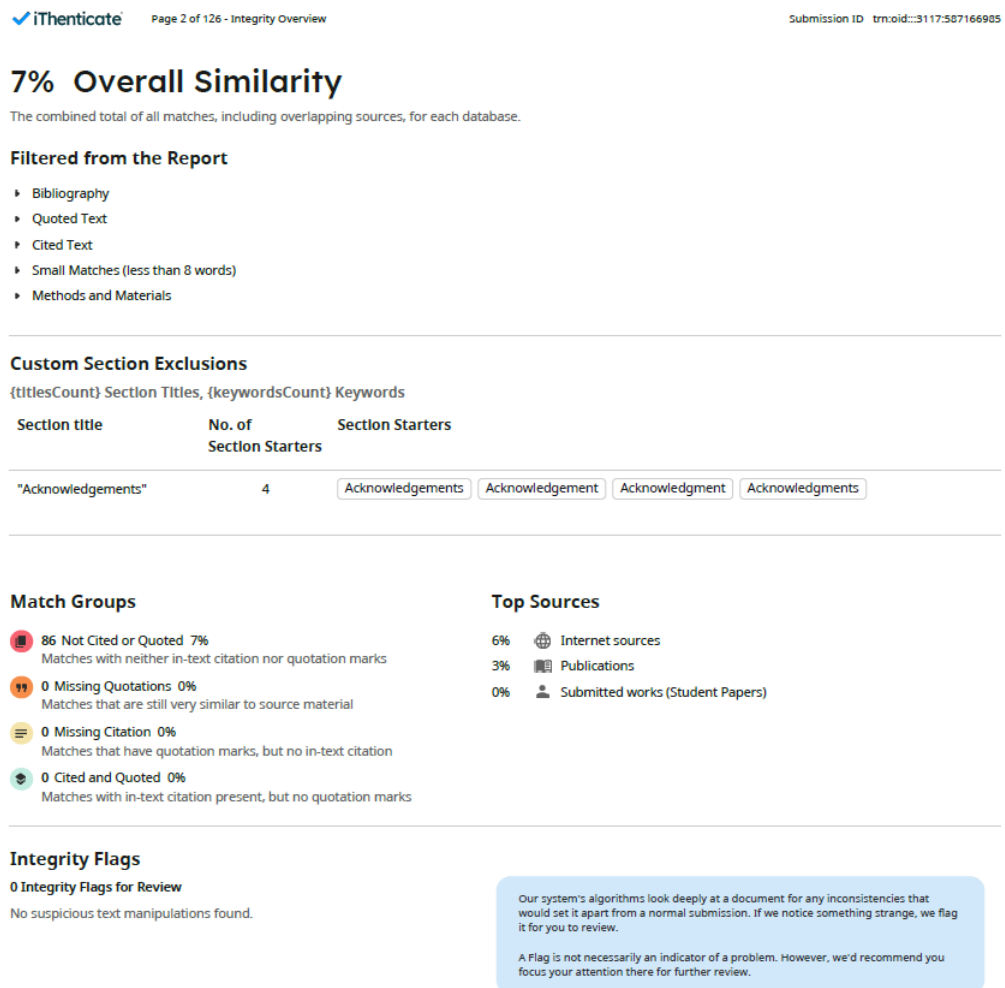


Figure 8-1: Summary Page of Plagiarism Report showing similarity index

Match Groups

-  **86 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 3%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	suramyapokharel.com.np	1%
2	Internet	www.researchgate.net	<1%
3	Internet	arxiv.org	<1%
4	Internet	www.coursehero.com	<1%
5	Internet	imat.entermedschool.com	<1%
6	Internet	vdocuments.mx	<1%
7	Internet	it.overleaf.com	<1%
8	Publication	Alexander Novikov. "SKYNET 2023 Conception of the Artificial Super Intelligence ...	<1%
9	Internet	core.ac.uk	<1%
10	Internet	assets-eu.researchsquare.com	<1%

Figure 8-2: Plagiarism Report showing Top sources

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Figure 8-3: AI writing Overview

Appendix G: Student and Supervisor Consultation Form

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING, THAPATHALI CAMPUS
Department of Electronics and Computer Engineering

Student & Supervisor Consultation Form
(BCT Major Project, Batch 2078)

Project Title	Mathsader: Automated Grading and Feedback system for Multi-step Algebraic Math solution using T-OCR, symbolic Reasoning and Deepseek math
Group Members (Name & Roll Number)	Jesis Upadhyay THA078BCT017 Prajwal Chaudhary THA078BCT028 Parushottam Gajurel THA078BCT032 Sagar Bikram Adhikari THA078BCT037
Name of Supervisor	Asst. Prof. Suwarna Lingden

S.N.	Brief Summary of Discussion Agenda	Date	Supervisor Signature
1	The feasibility and road-map of the project Suggested necessary changes in the report	2082/03/27	<i>[Signature]</i>
2	Checked the incorporations of previous date feedback and allowed to proceed for proposal defense	2082/03/28	<i>[Signature]</i>
3	Checked the progress of the project & allowed for incorporation of API based Mathpix for OCR purpose.	2082/03/28	<i>[Signature]</i>
4	Saw the improvements of the project & changes suggested for elaboration of the discussion part	2082/03/30	<i>[Signature]</i>
5	The reports is checked and suggested for the improvement of the flaws	2082/04/7	<i>[Signature]</i>
6	Checked the status of the project progress and suggested required changes	2082/04/8	<i>[Signature]</i>
7	Reviewed slides and videos for presentation	2082/04/9	<i>[Signature]</i>
8			
9			
10			

Date of Approval

Signature of Supervisor

At least **TWO** consultation is required before **Final Proposal Defense**
 At least **FIVE** consultations are required **BEFORE Midterm** and **TEN** consultations for **FINAL**

Figure 8-4: Student and Supervisor Consultation Form (Part-A)

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING, THAPATHALI CAMPUS
Department of Electronics and Computer Engineering

Student & Supervisor Consultation Form
(BCT Major Project, Batch 2078)

Project Title	Automated Marking and feedback system for Multi-step Math Solution using symbolic reasoning and Deepseek Math
Group Members (Name & Roll Number)	Jesis Upadhyaya (THA078BCT037) Prasidul Chaudhary (THA078BCT028) Purushottam Gajurel (THA078BCT032) Sagar Bikram Adhikari (THA078BCT028)
Name of Supervisor	Er. Sudarna Lingden

S.N.	Brief Summary of Discussion Agenda	Date	Supervisor Signature
1	Reviewed and suggested the changes in the report	2082/08/25	<i>[Signature]</i>
2	Recommended for progress defense	2082/09/01	<i>[Signature]</i>
3	Discussed the feedback given during progress defense and suggested to explore the mathematical rigor	2082/09/07	<i>[Signature]</i>
4	Checked the implementation of mathematical formulations and suggested necessary changes	2082/09/19	<i>[Signature]</i>
5	Dataset and subpic generation and fine-tuning; gave other feedback accordingly	2082/09/29	<i>[Signature]</i>
6	Discussion on incorporation of feedback from the progress defense; assigned tasks	2082/10/08	<i>[Signature]</i>
7	Checked the progress of assigned tasks; suggested necessary changes	2082/10/15	<i>[Signature]</i>
8	Evaluated the results and recommended necessary changes	2082/11/15	<i>[Signature]</i>
9	Documentation preparation checked and asked for necessary incorporation	2082/12/2	<i>[Signature]</i>
10	Final check of the report and asked for necessary changes	2082/12/2	<i>[Signature]</i>

2082/12/15

Date of Approval

[Signature]

Signature of Supervisor

At least **TWO** consultation is required before **Final Proposal Defense**
At least **FIVE** consultations are required **BEFORE** Midterm and **TEN** consultations for **FINAL**

Figure 8-5: Student and Supervisor Consultation Form (Part-B)

References

- [1] A. S. Lan, D. Vats, A. E. Waters, and R. G. Baraniuk, “Mathematical language processing: Automatic grading and feedback for open response mathematical questions,” 2015.
- [2] J. Kadupitiya, S. Ranathunga, and G. Dias, “Automated assessment of multi-step answers for mathematical word problems,” in *2016 Sixteenth International Conference on Advances in ICT for Emerging Regions (ICTer)*, pp. 66–71, IEEE, 2016.
- [3] W. Xie, J. Niu, C. J. Xue, and N. Guan, “Grade like a human: Rethinking automated assessment with large language models,” 2024.
- [4] R. M. Balakrishnan, P. B. Pati, R. P. Singh, S. S, and P. Kumar, “Fine-tuned t5 for auto-grading of quadratic equation problems,” *Procedia Computer Science*, vol. 235, pp. 2178–2186, 2024. International Conference on Machine Learning and Data Engineering (ICMLDE 2023).
- [5] SymPy Development Team, “SymPy: A python library for symbolic mathematics.” <https://www.sympy.org/>, 2024.
- [6] Hugging Face, “T5 Model Documentation.” https://huggingface.co/transformers/v3.0.2/model_doc/t5.html, 2025. [Online; accessed 2-June-2025].
- [7] P. Deepak, R. Rohan, R. Rohith, and R. R., “Nlp and ocr based automatic answer script evaluation system,” *International Journal of Computer Applications*, vol. 186, pp. 22–27, Sep 2024.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023.
- [9] M. Abdin, J. Aneja, S. B, C. Mendes, W. Chen, A. Giorno, R. Eldan, S. Gopi, S. Gunasekar, M. Javaheripi, P. Kauffmann, Y. T. Lee, Y. Li, A. Nguyen, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, M. Santacrose, and Y. Zhang, “Phi-2: The surprising power of small language models,” 12 2023.
- [10] K. Wang, H. Ren, A. Zhou, Z. Lu, S. Luo, W. Shi, R. Zhang, L. Song, M. Zhan, and H. Li, “Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning,” 2023.

- [11] A. Forootani, “A survey on mathematical reasoning and optimization with large language models,” 2025.
- [12] C. Cohn, N. Hutchins, T. Le, and G. Biswas, “A chain-of-thought prompting approach with llms for evaluating students’ formative assessment responses in science,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, p. 23182–23190, Mar. 2024.
- [13] E. Latif and X. Zhai, “Fine-tuning chatgpt for automatic scoring,” 2023.
- [14] Lightman, Kosaraju, Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” *arXiv preprint arXiv:2305.20050*, 2023.
- [15] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, “Trocr: Transformer-based optical character recognition with pre-trained models,” 2022.
- [16] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” 2024.
- [17] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [18] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the math dataset,” *NeurIPS*, 2021.
- [19] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, K. Lu, M. Xue, R. Lin, T. Liu, X. Ren, and Z. Zhang, “Qwen2.5-math technical report: Toward mathematical expert model via self-improvement,” 2024.
- [20] N. Shazeer, “Glu variants improve transformer,” 2020.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021.
- [22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language

models,” 2023.

- [23] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, “SymPy: symbolic computing in python,” *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017.
- [24] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *Journal of Machine Learning Research*, vol. 12, no. 61, pp. 2121–2159, 2011.
- [25] T. Tieleman, “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,” 2012.
- [26] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017.